Optimization-based Invariant Generation for Cyber-Physical Systems

Hao Wu

Supervisor: Prof. Naijun Zhan

Institute of Software, Chinese Academy of Sciences (ISCAS)
University of Chinese Academy of Sciences (UCAS)

PhD Defense
May 7, 2026

Contents

What is an invariant?

In mathematics, an invariant is a property of a mathematical object which remains unchanged after operations or transformations.

In computer science, an invariant is a logical assertion that is always held to be true during a certain phase of execution of a computer program.

— *from Wikipedia*

What is an invariant?

In mathematics, an invariant is a property of a mathematical object which remains unchanged after operations or transformations.

In computer science, an invariant is a logical assertion that is always held to be true during a certain phase of execution of a computer program.

— from Wikipedia

Verifying Program Correctness

- Consider a piece of program:

$$
\begin{aligned}
&x \leftarrow 2\\
&\text{while } x \leq 100 \text{ do}\\
&\quad x \leftarrow 2x - 1\\
&\text{end}
\end{aligned}
$$

- Question: How to reason about the correctness of a program?

To prove that $x > 0$ always holds.

- Raised by von Neumann (1946), Turing (1949), and McCarthy (1960).
- Formalized by Floyd, Hoare, and Naur in 1960s. $\Longrightarrow$ **Hoare logic**.

Verifying Program Correctness

- Consider a piece of program:

$$
\begin{aligned}
&x \leftarrow 2\\
&\text{while } x \leq 100 \text{ do}\\
&\quad x \leftarrow 2x - 1\\
&\text{end}
\end{aligned}
$$

- Question: How to reason about the correctness of a program?

To prove that $x > 0$ always holds.

- Raised by von Neumann (1946), Turing (1949), and McCarthy (1960).
- Formalized by Floyd, Hoare, and Naur in 1960s. $\Longrightarrow$ **Hoare logic**.

Hoare Logic

- **Hoare Logic**: to insert logical assertions into programs:

$\{\top\}$

$x \leftarrow 2$

$\{x = 2\}$ (condition at this location)

while $x \leq 100$ do $\{\ ?\ \}$

$\quad x \leftarrow 2x - 1$

end

$\{x > 0\}$

- Main Challenge: How to reason about loops?
- Hoare logic: **Loop Inductive Invariants**.

Hoare Logic

- **Hoare Logic**: to insert logical assertions into programs:

$\{\top\}$

$x \leftarrow 2$

$\{x = 2\}$ (condition at this location)

while $x \leq 100$ do $\{\ ?\ \}$

$\quad x \leftarrow 2x - 1$

end

$\{x > 0\}$

- Main Challenge: How to reason about loops?
- Hoare logic: **Loop Inductive Invariants**.

Hoare Logic

- **Hoare Logic**: to insert logical assertions into programs:

$\{\top\}$
$x \leftarrow 2$
$\{x = 2\}$ (condition at this location)
while $x \leq 100$ do $\{\ ?\ \}$
  $x \leftarrow 2x - 1$
end
$\{x > 0\}$

- Main Challenge: How to reason about loops?
- Hoare logic: **Loop Inductive Invariants**.

Invariants and Inductive Invariants

$\{x = 2\}$

while $x \leq 100$ do $\{\ ?\ \}$

$\quad x \leftarrow 2x - 1$

end

$\{x > 0\}$

- **Loop Invariant**: An assertion at the loop entrance that holds whenever the program reaches that point (e.g. $x > 0$).
- **Loop Inductive Invariant**: A loop invariant that is inductive, i.e., if it holds in one iteration, then it still holds in the next iteration. (e.g. $x \geq 1$).

$$x \geq 1 \land x' = 2x - 1 \land x \leq 100 \implies x' \geq 1$$

- By "invariants", we always mean inductive invariants.

Invariants and Inductive Invariants

$\{x = 2\}$

while $x \leq 100$ do $\{\ ?\ \}$

$x \leftarrow 2x - 1$

end

$\{x > 0\}$

- **Loop Invariant**: An assertion at the loop entrance that holds whenever the program reaches that point (e.g. $x > 0$).
- **Loop Inductive Invariant**: A loop invariant that is inductive, i.e., if it holds in one iteration, then it still holds in the next iteration. (e.g. $x \geq 1$).

$$x \geq 1 \land x' = 2x - 1 \land x \leq 100 \implies x' \geq 1$$

- By "invariants", we always mean inductive invariants.

Invariants and Inductive Invariants

$\{x = 2\}$

while $x \leq 100$ do $\{\ ?\ \}$

$x \leftarrow 2x - 1$

end

$\{x > 0\}$

- **Loop Invariant**: An assertion at the loop entrance that holds whenever the program reaches that point (e.g. $x > 0$).
- **Loop Inductive Invariant**: A loop invariant that is inductive, i.e., if it holds in one iteration, then it still holds in the next iteration. (e.g. $x \geq 1$).

$$x \geq 1 \land x' = 2x - 1 \land x \leq 100 \implies x' \geq 1$$

- By "invariants", we always mean inductive invariants.

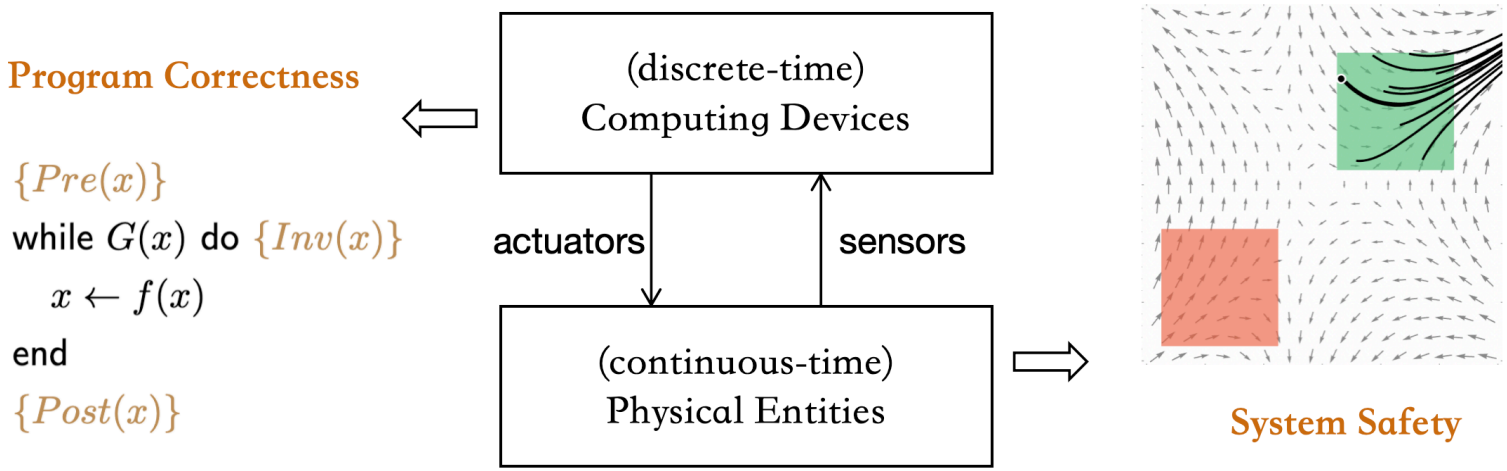
Verification Conditions

$\{Pre(x)\}$
while $G(x)$ do $\{Inv(x)\}$
  $x \leftarrow f(x)$
end
$\{Post(x)\}$

- Proving loop correctness is reduced to finding an suitable invariant:

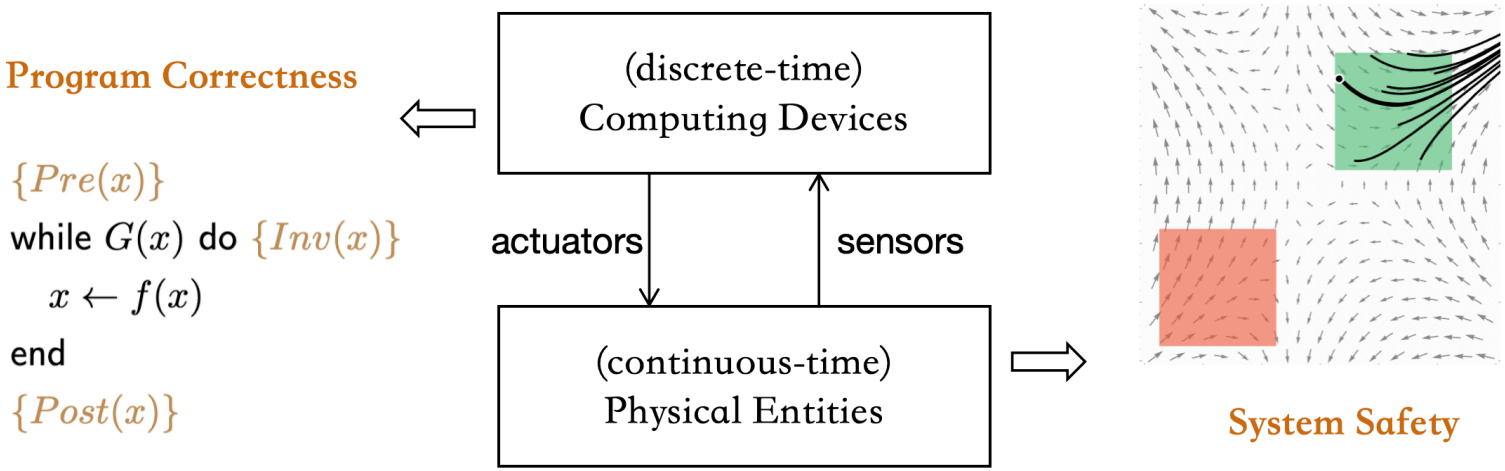
$$\forall x.\ Pre(x) \implies Inv(x)$$
$$\forall x.\ Inv(x) \wedge G(x) \implies Inv(f(x))$$
$$\forall x.\ Inv(x) \wedge \neg G(x) \implies Post(x)$$

Safety of Cyber-Physical Systems (CPS)



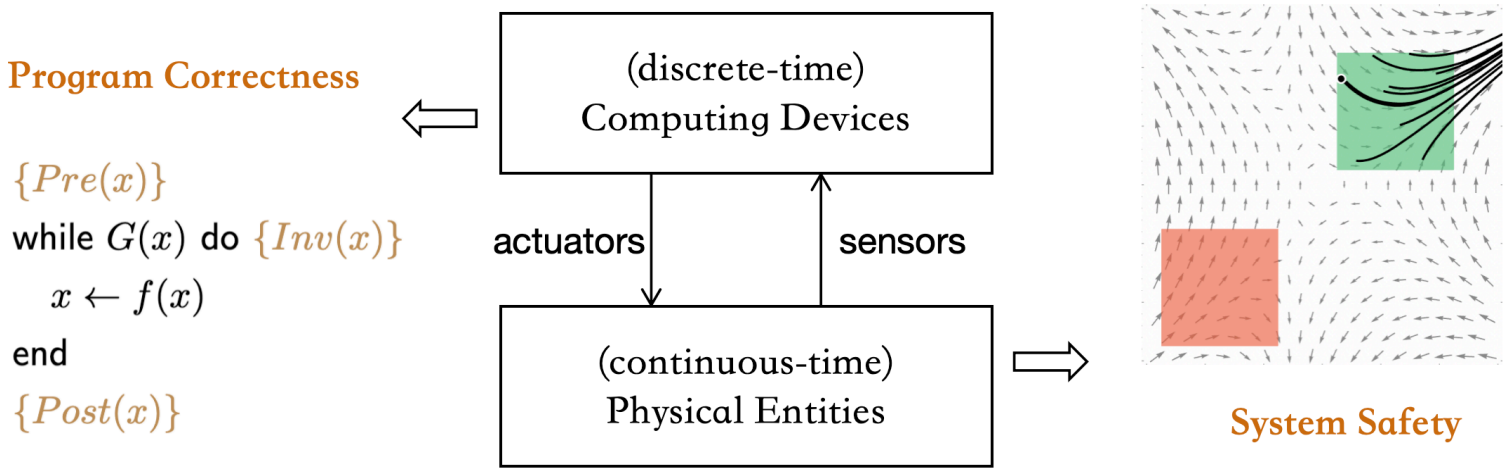
- **CPS** = embedded system + deep integration of computing, communication, and control
- **Safety**: the system will not evolve into bad states.
- Focus of this thesis: a unifed framework for both discrete and continuous dynamics.

Safety of Cyber-Physical Systems (CPS)



- **CPS** = embedded system + deep integration of computing, communication, and control
- **Safety**: the system will not evolve into bad states.
- Focus of this thesis: a unifed framework for both discrete and continuous dynamics.

Safety of Cyber-Physical Systems (CPS)



- **CPS** = embedded system + deep integration of computing, communication, and control
- **Safety**: the system will not evolve into bad states.
- Focus of this thesis: a unifed framework for both discrete and continuous dynamics.

Extending to Continuous-Time Dynamical Systems

System dynamics given by Ordinary Differential Equations (ODEs):

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}), \quad \boldsymbol{x} \in \mathcal{X}_d$$

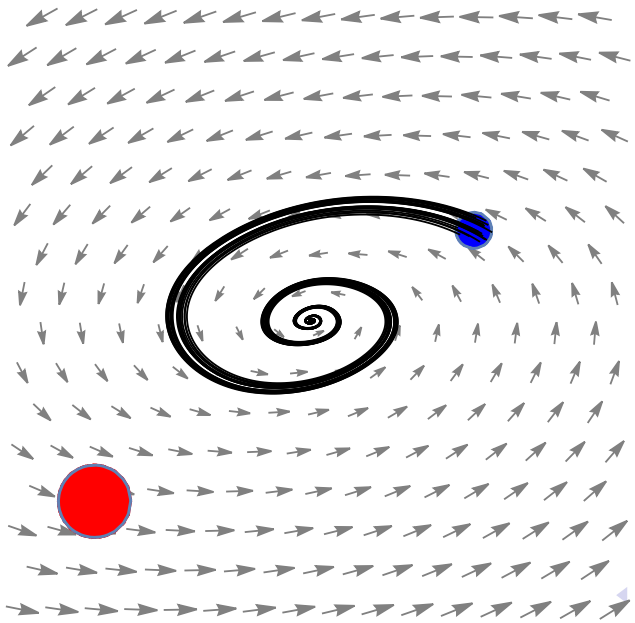
- $\boldsymbol{f}: \mathbb{R}^n \to \mathbb{R}^n$ is a locally Lipschitz continuous vector field.
- Unique trajectory $\xi_{\boldsymbol{x}_0} : \mathbb{R}_{\geq 0} \to \mathbb{R}^n$ for any initial state.

Safety Verification Problem

Safety Verification Problem of ODE Systems: Given domain $\mathcal{X}_d \subseteq \mathbb{R}^n$, initial set $\mathcal{X}_i \subset \mathcal{X}_d$, and unsafe set $\mathcal{X}_u \subset \mathcal{X}_d$ such that $\mathcal{X}_u \cap \mathcal{X}_i = \emptyset$. Prove that

$$\mathcal{X}_u \text{ is not reachable from any state in } \mathcal{X}_i.$$

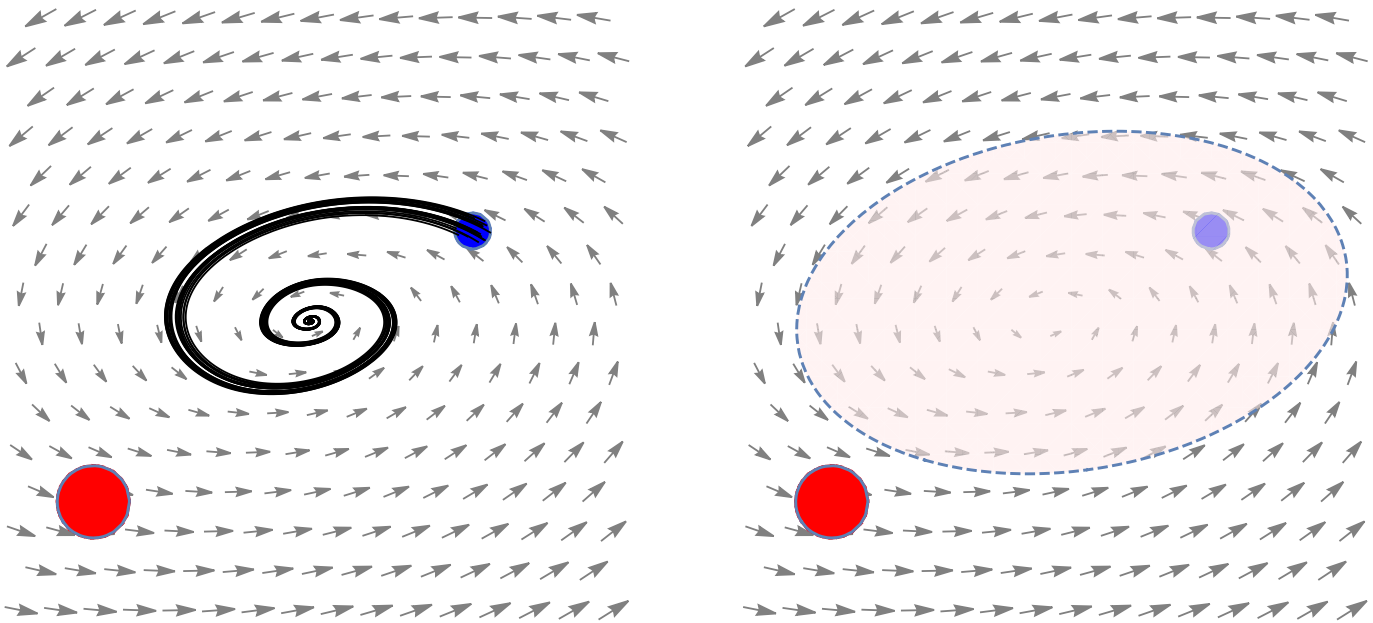
- $\mathcal{X}_i$: pre-condition.
- $\mathcal{X}_u$: negation of post-condition.
- $\mathcal{X}_d$: variable range



Positive Invariants

- Analogous to **inductive invariants** for programs.
- A **positive/differential invariant** is a subset of states $\Omega \subseteq \mathcal{X}_d$ s.t. any trajectory starting from Ω stays within Ω forever, i.e.,

$$\forall \mathbf{x}_0 \in \Omega, \forall t \in \mathbb{R}_{\geq 0}.\ \xi_{\mathbf{x}_0}(t) \in \Omega.$$



Research Question

- The invariant generation problem is undecidable in general.
  - Because finding the exact set of reachable states is undecidable.
- Undecidable even for affine programs with affine guards [Müller-Olm et al., 2004].
- Undecidable even for polynomial ODE [Graça et al., 2008].

Research Question

How to generate certain type of invariants for certain type of systems?

Research Question

- The invariant generation problem is undecidable in general.
  - Because finding the exact set of reachable states is undecidable.
- Undecidable even for affine programs with affine guards [Müller-Olm et al., 2004].
- Undecidable even for polynomial ODE [Graça et al., 2008].

Research Question

How to generate certain type of invariants for certain type of systems?

Research Question

- The invariant generation problem is undecidable in general.
  - Because finding the exact set of reachable states is undecidable.
- Undecidable even for affine programs with affine guards [Müller-Olm et al., 2004].
- Undecidable even for polynomial ODE [Graça et al., 2008].

Research Question

How to generate certain type of invariants for certain type of systems?

Research Question

- The invariant generation problem is undecidable in general.
  - Because finding the exact set of reachable states is undecidable.
- Undecidable even for affine programs with affine guards [Müller-Olm et al., 2004].
- Undecidable even for polynomial ODE [Graça et al., 2008].

Research Question

How to generate certain type of invariants for certain type of systems?

Invariant Generation Methods

- **Programs**:
  - Constraint Solving
  - Abstract Interpretation
  - Recursive Relation
  - Counter-Example Guided Inductive Synthesis
  - Machine Learning
- **Continuous-time systems**:
  - Constraint Solving
  - Differential Algebra
  - Machine Learning
- **Advantages** of Constraint Solving:
  - Works for both discrete- and continuous-time systems.
  - Offers a relative completeness guarantee.
  - Uses efficient constraint-solving and optimization tools.

Invariant Generation Methods

- **Programs**:
  - Constraint Solving
  - Abstract Interpretation
  - Recursive Relation
  - Counter-Example Guided Inductive Synthesis
  - Machine Learning
- **Continuous-time systems**:
  - Constraint Solving
  - Differential Algebra
  - Machine Learning
- **Advantages** of Constraint Solving:
  - Works for both discrete- and continuous-time systems.
  - Offers a relative completeness guarantee.
  - Uses efficient constraint-solving and optimization tools.

Invariant Generation Methods

- **Programs**:
  - Constraint Solving
  - Abstract Interpretation
  - Recursive Relation
  - Counter-Example Guided Inductive Synthesis
  - Machine Learning
- **Continuous-time systems**:
  - Constraint Solving
  - Differential Algebra
  - Machine Learning
- **Advantages** of Constraint Solving:
  - Works for both discrete- and continuous-time systems.
  - Offers a relative completeness guarantee.
  - Uses efficient constraint-solving and optimization tools.

Assumptions

(1) Variables are $\mathbb{R}$-valued.
- No integer/Boolean variables. No other data structures.

(2) Functions are polynomial:
- program updates and ODE dynamics.

(3) Predicates are basic semialgebraic sets
- target invariants, pre- and post-conditions, initial, unsafe, and domain regions
- defined by a conjunction of polynomial inequalities

$$S = \{\boldsymbol{x} \in \mathbb{R}^n \mid p_1(\boldsymbol{x}) \geq 0, \ldots, p_m(\boldsymbol{x}) \geq 0\} \quad \text{with } p_i \in \mathbb{R}[\boldsymbol{x}].$$

Assumptions

(1) Variables are $\mathbb{R}$-valued.
- No integer/Boolean variables. No other data structures.

(2) Functions are polynomial:
- program updates and ODE dynamics.

(3) Predicates are basic semialgebraic sets
- target invariants, pre- and post-conditions, initial, unsafe, and domain regions
- defined by a conjunction of polynomial inequalities

$$S = \{\boldsymbol{x} \in \mathbb{R}^n \mid p_1(\boldsymbol{x}) \geq 0, \ldots, p_m(\boldsymbol{x}) \geq 0\} \quad \text{with } p_i \in \mathbb{R}[\boldsymbol{x}].$$

Assumptions

(1) Variables are $\mathbb{R}$-valued.
- No integer/Boolean variables. No other data structures.

(2) Functions are polynomial:
- program updates and ODE dynamics.

(3) Predicates are basic semialgebraic sets
- target invariants, pre- and post-conditions, initial, unsafe, and domain regions
- defined by a conjunction of polynomial inequalities

$$S = \{\boldsymbol{x} \in \mathbb{R}^n \mid p_1(\boldsymbol{x}) \geq 0, \ldots, p_m(\boldsymbol{x}) \geq 0\} \quad \text{with } p_i \in \mathbb{R}[\boldsymbol{x}].$$

Constraint-based synthesis: inductive invariants

1. **Template Setting**: set a parameterized template to represent the invariant $I(\boldsymbol{x}) \leq 0$ where

$$I(\mathbf{x}; \mathbf{a}) = a_1 x_1^2 + a_2 x_1 x_2 + a_3 x_2^2 + a_4 x_1 + a_5 x_2 + a_6 \in \mathbb{R}[x_1, x_2].$$

2. **Condition Encoding**: encode the invariant conditions into constraints.

$$\text{Substitute } Inv(\boldsymbol{x}) \text{ by } I(\boldsymbol{x}; \boldsymbol{a}) \leq 0$$

3. **Constraint Solving**: solve the constraints for the parameters $\boldsymbol{a}$:
   1. $\forall \mathbf{x}.\ \text{Pre}(\boldsymbol{x}) \implies I(\mathbf{x}; \mathbf{a}) \leq 0,$
   2. $\forall \mathbf{x}.\ I(\mathbf{x}; \mathbf{a}) \leq 0 \wedge g(\boldsymbol{x}) \leq 0 \implies I(f(\mathbf{x}); \mathbf{a}) \leq 0,$
   3. $\forall \mathbf{x}.\ I(\mathbf{x}; \mathbf{a}) \leq 0 \wedge g(\boldsymbol{x}) > 0 \implies \text{Post}(\boldsymbol{x}),$

   sufficient and necessary condition for $I(\boldsymbol{x}, \boldsymbol{a}) \leq 0$ to be an inductive invariant.

Constraint-based synthesis: inductive invariants

1. **Template Setting**: set a parameterized template to represent the invariant $I(\boldsymbol{x}) \leq 0$ where

$$I(\mathbf{x}; \mathbf{a}) = a_1 x_1^2 + a_2 x_1 x_2 + a_3 x_2^2 + a_4 x_1 + a_5 x_2 + a_6 \in \mathbb{R}[x_1, x_2].$$

2. **Condition Encoding**: encode the invariant conditions into constraints.

Substitute $Inv(\boldsymbol{x})$ by $I(\boldsymbol{x}; \mathbf{a}) \leq 0$

3. **Constraint Solving**: solve the constraints for the parameters $\boldsymbol{a}$:
   1. $\forall \mathbf{x}.\ \mathrm{Pre}(\boldsymbol{x}) \implies I(\mathbf{x}; \mathbf{a}) \leq 0,$
   2. $\forall \mathbf{x}.\ I(\mathbf{x}; \mathbf{a}) \leq 0 \wedge g(\boldsymbol{x}) \leq 0 \implies I(f(\boldsymbol{x}); \mathbf{a}) \leq 0,$
   3. $\forall \mathbf{x}.\ I(\mathbf{x}; \mathbf{a}) \leq 0 \wedge g(\boldsymbol{x}) > 0 \implies \mathrm{Post}(\boldsymbol{x}),$

   sufficient and necessary condition for $I(\boldsymbol{x}, \boldsymbol{a}) \leq 0$ to be an inductive invariant.

Constraint-based synthesis: inductive invariants

1. **Template Setting**: set a parameterized template to represent the invariant $I(\boldsymbol{x}) \leq 0$ where

$$I(\mathbf{x}; \mathbf{a}) = a_1 x_1^2 + a_2 x_1 x_2 + a_3 x_2^2 + a_4 x_1 + a_5 x_2 + a_6 \in \mathbb{R}[x_1, x_2].$$

2. **Condition Encoding**: encode the invariant conditions into constraints.

$$\text{Substitute } Inv(\boldsymbol{x}) \text{ by } I(\boldsymbol{x}; \mathbf{a}) \leq 0$$

3. **Constraint Solving**: solve the constraints for the parameters $\boldsymbol{a}$:
   1. $\forall \boldsymbol{x}.\ \text{Pre}(\boldsymbol{x}) \implies I(\boldsymbol{x}; \mathbf{a}) \leq 0,$
   2. $\forall \boldsymbol{x}.\ I(\boldsymbol{x}; \mathbf{a}) \leq 0 \wedge g(\boldsymbol{x}) \leq 0 \implies I(f(\boldsymbol{x}); \mathbf{a}) \leq 0,$
   3. $\forall \boldsymbol{x}.\ I(\boldsymbol{x}; \mathbf{a}) \leq 0 \wedge g(\boldsymbol{x}) > 0 \implies \text{Post}(\boldsymbol{x}),$

   sufficient and necessary condition for $I(\boldsymbol{x}, \mathbf{a}) \leq 0$ to be an inductive invariant.

Constraint-based synthesis: positive invariants

- Set a template $B(\boldsymbol{x};\boldsymbol{a}) \leq 0$ for the positive invariant.
  - B is called a **Barrier Certificate (BC)** [Prajna et al., 2004].
- Encode the sufficient conditions for positive invariance:
  1. $\forall \mathbf{x}.\ \mathbf{x} \in \mathcal{X}_i \implies B(\mathbf{x};\mathbf{a}) \leq 0,$
  2. $\forall \mathbf{x}.\ \mathbf{x} \in \mathcal{X}_d \wedge B(\mathbf{x};\mathbf{a}) = 0 \implies \mathfrak{L}_{\boldsymbol{f}} B(\mathbf{x};\mathbf{a}) < 0,$
  3. $\forall \mathbf{x}.\ \mathbf{x} \in \mathcal{X}_u \implies B(\mathbf{x};\mathbf{a}) > 0.$

  where $\mathfrak{L}_{\boldsymbol{f}} B \hat{=} \langle \nabla B, \boldsymbol{f} \rangle$ is the Lie derivative.
- Necessity requires higher-order Lie derivatives [Liu et al., 2011].

Constraint-based synthesis: positive invariants

- Set a template $B(\boldsymbol{x};\boldsymbol{a}) \leq 0$ for the positive invariant.
  - B is called a **Barrier Certificate (BC)** [Prajna et al., 2004].
- Encode the sufficient conditions for positive invariance:
  1. $\forall \mathbf{x}.\ \mathbf{x} \in \mathcal{X}_i \implies B(\mathbf{x};\mathbf{a}) \leq 0,$
  2. $\forall \mathbf{x}.\ \mathbf{x} \in \mathcal{X}_d \wedge B(\mathbf{x};\mathbf{a}) = 0 \implies \mathfrak{L}_{\boldsymbol{f}} B(\mathbf{x};\mathbf{a}) < 0,$
  3. $\forall \mathbf{x}.\ \mathbf{x} \in \mathcal{X}_u \implies B(\mathbf{x};\mathbf{a}) > 0.$

  where $\mathfrak{L}_{\boldsymbol{f}} B \hat{=} \langle \nabla B, \boldsymbol{f} \rangle$ is the Lie derivative.
- Necessity requires higher-order Lie derivatives [Liu et al., 2011].

From the view of constraint solving

- The synthesis problems ask to solve some quantified constraints:

$$\forall \boldsymbol{x}.\ I(\boldsymbol{x};\boldsymbol{a}) \le 0 \wedge g(\boldsymbol{x}) \le 0 \implies I(f(\boldsymbol{x});\boldsymbol{a}) \le 0 \qquad \text{(inductive)}$$

$$\forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_d \wedge B(\boldsymbol{x};\boldsymbol{a}) = 0 \implies \mathfrak{L}_{\boldsymbol{f}} B(\boldsymbol{x};\boldsymbol{a}) < 0 \qquad \text{(differential)}$$

- Decidable by **Quantifier Elimination** $(\exists \boldsymbol{a}, \forall \boldsymbol{x})$.
- Directly dealing with $\exists\forall$ is difficult for SMT and optimization tools.
- A (standard) solution: use algebraic representation theorems (e.g. **Putinar's Positivstellensatz**) to remove $\forall$.

From the view of constraint solving

- The synthesis problems ask to solve some quantified constraints:

$$\forall \boldsymbol{x}.\ I(\boldsymbol{x};\boldsymbol{a}) \le 0 \wedge g(\boldsymbol{x}) \le 0 \implies I(f(\boldsymbol{x});\boldsymbol{a}) \le 0 \qquad \text{(inductive)}$$

$$\forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_d \wedge B(\boldsymbol{x};\boldsymbol{a}) = 0 \implies \mathfrak{L}_{\boldsymbol{f}} B(\boldsymbol{x};\boldsymbol{a}) < 0 \qquad \text{(differential)}$$

- Decidable by **Quantifier Elimination** $(\exists \boldsymbol{a}, \forall \boldsymbol{x})$.
- Directly dealing with $\exists\forall$ is difficult for SMT and optimization tools.
- A (standard) solution: use algebraic representation theorems (e.g. **Putinar's Positivstellensatz**) to remove $\forall$.

From the view of constraint solving

- The synthesis problems ask to solve some quantified constraints:

$$\forall \boldsymbol{x}.\ I(\boldsymbol{x};\boldsymbol{a}) \le 0 \wedge g(\boldsymbol{x}) \le 0 \implies I(f(\boldsymbol{x});\boldsymbol{a}) \le 0 \qquad \text{(inductive)}$$

$$\forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_d \wedge B(\boldsymbol{x};\boldsymbol{a}) = 0 \implies \mathfrak{L}_{\boldsymbol{f}} B(\boldsymbol{x};\boldsymbol{a}) < 0 \qquad \text{(differential)}$$

- Decidable by **Quantifier Elimination** $(\exists \boldsymbol{a}, \forall \boldsymbol{x})$.
- Directly dealing with $\exists\forall$ is difficult for SMT and optimization tools.
- A (standard) solution: use algebraic representation theorems (e.g. **Putinar's Positivstellensatz**) to remove $\forall$.

From the view of constraint solving

- The synthesis problems ask to solve some quantified constraints:

$$\forall \boldsymbol{x}.\ I(\boldsymbol{x};\boldsymbol{a}) \le 0 \wedge g(\boldsymbol{x}) \le 0 \implies I(f(\boldsymbol{x});\boldsymbol{a}) \le 0 \qquad \text{(inductive)}$$

$$\forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_d \wedge B(\boldsymbol{x};\boldsymbol{a}) = 0 \implies \mathfrak{L}_{\boldsymbol{f}} B(\boldsymbol{x};\boldsymbol{a}) < 0 \qquad \text{(differential)}$$

- Decidable by **Quantifier Elimination** ($\exists \boldsymbol{a}, \forall \boldsymbol{x}$).
- Directly dealing with $\exists\forall$ is difficult for SMT and optimization tools.
- A (standard) solution: use algebraic representation theorems (e.g. **Putinar's Positivstellensatz**) to remove $\forall$.

Tools: SOS polynomials

- A polynomial $\sigma(\boldsymbol{x})$ is **Sum-of-Squares** (SOS) if it can written as

$$\sigma(\boldsymbol{x}) = \sum_{i=1}^{n} p_i^2(\boldsymbol{x}) \in \mathbb{R}[\boldsymbol{x}].$$

- $\Sigma[\boldsymbol{x}]$ denotes the set of SOS polynomials.
- A polynomial $\sigma(\boldsymbol{x})$ is SOS iff the Gram matrix of σ is PSD.

$$\begin{aligned}
\sigma(x_1, x_2) &= 2x_1^4 + 2x_1^3x_2 - x_1^2x_2^2 + 5x_2^4 \\
&= \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}'}_{\text{basis}} \underbrace{\begin{pmatrix} 2 & -3 & 1 \\ -3 & 5 & 0 \\ 1 & 0 & 5 \end{pmatrix}}_{\text{Gram matrix}} \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}}_{\text{basis}} \\
&= \frac{1}{2}(2x_1^2 - 3x_2^3 + x_1x_2)^2 + \frac{1}{2}(x_2^2 + 3x_1x_2)^2
\end{aligned}$$

Tools: SOS polynomials

- A polynomial $\sigma(\boldsymbol{x})$ is **Sum-of-Squares** (SOS) if it can written as

$$\sigma(\mathbf{x}) = \sum_{i=1}^{n} p_i^2(\mathbf{x}) \in \mathbb{R}[\mathbf{x}].$$

- $\Sigma[\mathbf{x}]$ denotes the set of SOS polynomials.
- A polynomial $\sigma(\boldsymbol{x})$ is SOS iff the Gram matrix of σ is PSD.

$$\begin{aligned}\sigma(x_1, x_2) &= 2x_1^4 + 2x_1^3x_2 - x_1^2x_2^2 + 5x_2^4 \\ &= \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}'}_{\text{basis}} \underbrace{\begin{pmatrix} 2 & -3 & 1 \\ -3 & 5 & 0 \\ 1 & 0 & 5 \end{pmatrix}}_{\text{Gram matrix}} \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}}_{\text{basis}} \\ &= \frac{1}{2}(2x_1^2 - 3x_2^3 + x_1x_2)^2 + \frac{1}{2}(x_2^2 + 3x_1x_2)^2\end{aligned}$$

Tools: SOS polynomials

- A polynomial $\sigma(\boldsymbol{x})$ is **Sum-of-Squares** (SOS) if it can written as

$$\sigma(\mathbf{x}) = \sum_{i=1}^{n} p_i^2(\mathbf{x}) \in \mathbb{R}[\mathbf{x}].$$

- $\Sigma[\mathbf{x}]$ denotes the set of SOS polynomials.
- A polynomial $\sigma(\boldsymbol{x})$ is SOS iff the Gram matrix of σ is PSD.

$$\begin{aligned}
\sigma(x_1, x_2) &= 2x_1^4 + 2x_1^3x_2 - x_1^2x_2^2 + 5x_2^4 \\
&= \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}'}_{\text{basis}} \underbrace{\begin{pmatrix} 2 & -3 & 1 \\ -3 & 5 & 0 \\ 1 & 0 & 5 \end{pmatrix}}_{\text{Gram matrix}} \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}}_{\text{basis}} \\
&= \frac{1}{2}(2x_1^2 - 3x_2^3 + x_1x_2)^2 + \frac{1}{2}(x_2^2 + 3x_1x_2)^2
\end{aligned}$$

Tools: SOS polynomials

- A polynomial $\sigma(\boldsymbol{x})$ is **Sum-of-Squares** (SOS) if it can written as

$$\sigma(\mathbf{x}) = \sum_{i=1}^{n} p_i^2(\mathbf{x}) \in \mathbb{R}[\mathbf{x}].$$

- $\Sigma[\mathbf{x}]$ denotes the set of SOS polynomials.
- A polynomial $\sigma(\boldsymbol{x})$ is SOS iff the Gram matrix of σ is PSD.

$$\begin{aligned}\sigma(x_1, x_2) &= 2x_1^4 + 2x_1^3x_2 - x_1^2x_2^2 + 5x_2^4 \\ &= \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}'}_{\text{basis}} \underbrace{\begin{pmatrix} 2 & -3 & 1 \\ -3 & 5 & 0 \\ 1 & 0 & 5 \end{pmatrix}}_{\text{Gram matrix}} \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}}_{\text{basis}} \\ &= \frac{1}{2}(2x_1^2 - 3x_2^3 + x_1x_2)^2 + \frac{1}{2}(x_2^2 + 3x_1x_2)^2\end{aligned}$$

Tools: Putinar's Positivstellensatz

- A basic semialgebraic set $S = \{x \mid p_i(x) \geq 0,\ i = 1, \ldots, m\}$.
- A **Quadratic Module** associated to S:

$$\mathbf{QM}(S) \triangleq \{\sigma_0 + \sum_{i=1}^{m} \sigma_i p_i \mid \sigma_i \in \Sigma[\mathbf{x}] \text{ for } 1 \leq i \leq m\} \subseteq \mathbb{R}[\mathbf{x}]$$

- A **QM** is **Archimedean** if $N - \|\mathbf{x}\|^2 \in \mathbf{QM}$ for some $N \in \mathbb{N}$.
  - This basically means that S is bounded.

Theorem (Putinar's Positivstellensatz)

Assume $\mathbf{QM}(S)$ is Archimedean. If $f \in \mathbb{R}[x]$ is strictly positive on S, then $f \in \mathbf{QM}(S)$, i.e.,

$$f(x) = \sigma_0(x) + \sum_{i=1}^{m} p_i(x) \cdot \sigma_i(x), \quad \sigma_i(x) \in \Sigma[x].$$

Tools: Putinar's Positivstellensatz

- A basic semialgebraic set $S = \{x \mid p_i(x) \geq 0,\ i = 1, \ldots, m\}$.
- A **Quadratic Module** associated to S:

$$\mathbf{QM}(S) \triangleq \{\sigma_0 + \sum_{i=1}^{m} \sigma_i p_i \mid \sigma_i \in \Sigma[\mathbf{x}] \text{ for } 1 \leq i \leq m\} \subseteq \mathbb{R}[\mathbf{x}]$$

- A **QM** is **Archimedean** if $N - \|\mathbf{x}\|^2 \in \mathbf{QM}$ for some $N \in \mathbb{N}$.
  - This basically means that S is bounded.

Theorem (Putinar's Positivstellensatz)

Assume $\mathbf{QM}(S)$ is Archimedean. If $f \in \mathbb{R}[x]$ is strictly positive on S, then $f \in \mathbf{QM}(S)$, i.e.,

$$f(x) = \sigma_0(x) + \sum_{i=1}^{m} p_i(x) \cdot \sigma_i(x), \quad \sigma_i(x) \in \Sigma[x].$$

Tools: Putinar's Positivstellensatz

- A basic semialgebraic set $S = \{x \mid p_i(x) \geq 0,\ i = 1, \ldots, m\}$.
- A **Quadratic Module** associated to S:

$$\mathbf{QM}(S) \triangleq \{\sigma_0 + \sum_{i=1}^{m} \sigma_i p_i \mid \sigma_i \in \Sigma[\mathbf{x}] \text{ for } 1 \leq i \leq m\} \subseteq \mathbb{R}[\mathbf{x}]$$

- A **QM** is **Archimedean** if $N - \|\mathbf{x}\|^2 \in \mathbf{QM}$ for some $N \in \mathbb{N}$.
  - This basically means that S is bounded.

Theorem (Putinar's Positivstellensatz)

Assume $\boldsymbol{QM}(S)$ *is Archimedean. If* $f \in \mathbb{R}[x]$ *is strictly positive on* S*, then* $f \in \boldsymbol{QM}(S)$*, i.e.,*

$$f(x) = \sigma_0(x) + \sum_{i=1}^{m} p_i(x) \cdot \sigma_i(x), \quad \sigma_i(x) \in \Sigma[x].$$

SOS Constraints for Inductive Invariants

- For simplicity, we assume that $\mathrm{Pre}(\boldsymbol{x})$ is $p(\boldsymbol{x}) \leq 0$ and $\mathrm{Post}(\boldsymbol{x})$ is $q(\boldsymbol{x}) \leq 0$:
  1. $\forall \mathbf{x}.\ p(\mathbf{x}) \leq 0 \implies I(\mathbf{x}; \mathbf{a}) \leq 0,$
  2. $\forall \mathbf{x}.\ I(\mathbf{x}; \mathbf{a}) \leq 0 \wedge g(\mathbf{x}) \leq 0 \implies I(f(\mathbf{x}); \mathbf{a}) \leq 0,$
  3. $\forall \mathbf{x}.\ I(\mathbf{x}; \mathbf{a}) \leq 0 \wedge g(\mathbf{x}) > 0 \implies q(\mathbf{x}) \leq 0.$
- By applying **Putinar's Psatz**, the above constraints are satisfiable if the following constraints are solvable, for any $\epsilon > 0$ arbitrarily small.

$$
\begin{aligned}
&\text{find} \quad \boldsymbol{a}, \boldsymbol{b} \\
&-I(\boldsymbol{x}; \boldsymbol{a}) + \epsilon = \sigma_1(\boldsymbol{x}; \boldsymbol{b}) + \sigma_2(\boldsymbol{x}; \boldsymbol{b}) \cdot (-p(\boldsymbol{x})), \\
&-I(f(\boldsymbol{x}); \boldsymbol{a}) + \epsilon = \sigma_3(\boldsymbol{x}; \boldsymbol{b}) + \sigma_4(\boldsymbol{x}; \boldsymbol{b}) \cdot (-g(\boldsymbol{x})) + \sigma_5(\boldsymbol{x}; \boldsymbol{b}) \cdot (-I(\boldsymbol{x}; \boldsymbol{a})) \\
&-q(\boldsymbol{x}) + \epsilon = \sigma_6(\boldsymbol{x}; \boldsymbol{b}) + \sigma_7(\boldsymbol{x}; \boldsymbol{b}) \cdot g(\boldsymbol{x}) + \sigma_8(\boldsymbol{x}; \boldsymbol{b}) \cdot (-I(\boldsymbol{x}; \boldsymbol{a})) \\
&\sigma_1(\boldsymbol{x}; \boldsymbol{b}), \ldots, \sigma_8(\boldsymbol{x}; \boldsymbol{b}) \in \Sigma[\boldsymbol{x}]
\end{aligned}
$$

SOS Constraints for Inductive Invariants

- For simplicity, we assume that $\mathrm{Pre}(\boldsymbol{x})$ is $p(\boldsymbol{x}) \leq 0$ and $\mathrm{Post}(\boldsymbol{x})$ is $q(\boldsymbol{x}) \leq 0$:
  1. $\forall \mathbf{x}.\ p(\mathbf{x}) \leq 0 \implies I(\mathbf{x}; \mathbf{a}) \leq 0,$
  2. $\forall \mathbf{x}.\ I(\mathbf{x}; \mathbf{a}) \leq 0 \wedge g(\mathbf{x}) \leq 0 \implies I(f(\mathbf{x}); \mathbf{a}) \leq 0,$
  3. $\forall \mathbf{x}.\ I(\mathbf{x}; \mathbf{a}) \leq 0 \wedge g(\mathbf{x}) > 0 \implies q(\mathbf{x}) \leq 0.$
- By applying **Putinar's Psatz**, the above constraints are satisfiable if the following constraints are solvable, for any $\epsilon > 0$ arbitrarily small.

$$\text{find} \quad \mathbf{a}, \mathbf{b}$$

$$-I(\mathbf{x}; \mathbf{a}) + \epsilon = \sigma_1(\mathbf{x}; \mathbf{b}) + \sigma_2(\mathbf{x}; \mathbf{b}) \cdot (-p(\mathbf{x})),$$

$$-I(f(\mathbf{x}); \mathbf{a}) + \epsilon = \sigma_3(\mathbf{x}; \mathbf{b}) + \sigma_4(\mathbf{x}; \mathbf{b}) \cdot (-g(\mathbf{x})) + \sigma_5(\mathbf{x}; \mathbf{b}) \cdot (-I(\mathbf{x}; \mathbf{a}))$$

$$-q(\mathbf{x}) + \epsilon = \sigma_6(\mathbf{x}; \mathbf{b}) + \sigma_7(\mathbf{x}; \mathbf{b}) \cdot g(\mathbf{x}) + \sigma_8(\mathbf{x}; \mathbf{b}) \cdot (-I(\mathbf{x}; \mathbf{a}))$$

$$\sigma_1(\mathbf{x}; \mathbf{b}), \ldots, \sigma_8(\mathbf{x}; \mathbf{b}) \in \Sigma[\boldsymbol{x}]$$

SOS Constraints for Inductive Invariants

$$
\begin{aligned}
&\text{find} \quad \boldsymbol{a}, \boldsymbol{b} \\
-I(\boldsymbol{x}; \boldsymbol{a}) + \epsilon &= \sigma_1(\boldsymbol{x}; \boldsymbol{b}) + \sigma_2(\boldsymbol{x}; \boldsymbol{b}) \cdot (-p(\boldsymbol{x})), \\
-I(f(\boldsymbol{x}); \boldsymbol{a}) + \epsilon &= \sigma_3(\boldsymbol{x}; \boldsymbol{b}) + \sigma_4(\boldsymbol{x}; \boldsymbol{b}) \cdot (-g(\boldsymbol{x})) + \sigma_5(\boldsymbol{x}; \boldsymbol{b}) \cdot (-I(\boldsymbol{x}; \boldsymbol{a})) \\
-q(\boldsymbol{x}) + \epsilon &= \sigma_6(\boldsymbol{x}; \boldsymbol{b}) + \sigma_7(\boldsymbol{x}; \boldsymbol{b}) \cdot g(\boldsymbol{x}) + \sigma_8(\boldsymbol{x}; \boldsymbol{b}) \cdot (-I(\boldsymbol{x}; \boldsymbol{a})) \\
&\sigma_1(\boldsymbol{x}; \boldsymbol{b}), \ldots, \sigma_8(\boldsymbol{x}; \boldsymbol{b}) \in \Sigma[\boldsymbol{x}]
\end{aligned}
$$

- When σ_5, σ_8 are known, **Linear Matrix Inequalities (LMIs)** $\Longrightarrow$ convex, solved by **Semidefinite Programming (SDP)**.
- In general, **Bilinear Matrix Inequalities (BMIs)** $\Longrightarrow$ non-convex and NP-hard.
- Similar for positive invariants.

SOS Constraints for Inductive Invariants

$$
\begin{aligned}
&\text{find} \quad \boldsymbol{a}, \boldsymbol{b} \\
-I(\boldsymbol{x};\boldsymbol{a}) + \epsilon &= \sigma_1(\boldsymbol{x};\boldsymbol{b}) + \sigma_2(\boldsymbol{x};\boldsymbol{b}) \cdot (-p(\boldsymbol{x})), \\
-I(f(\boldsymbol{x});\boldsymbol{a}) + \epsilon &= \sigma_3(\boldsymbol{x};\boldsymbol{b}) + \sigma_4(\boldsymbol{x};\boldsymbol{b}) \cdot (-g(\boldsymbol{x})) + \sigma_5(\boldsymbol{x};\boldsymbol{b}) \cdot (-I(\boldsymbol{x};\boldsymbol{a})) \\
-q(\boldsymbol{x}) + \epsilon &= \sigma_6(\boldsymbol{x};\boldsymbol{b}) + \sigma_7(\boldsymbol{x};\boldsymbol{b}) \cdot g(\boldsymbol{x}) + \sigma_8(\boldsymbol{x};\boldsymbol{b}) \cdot (-I(\boldsymbol{x};\boldsymbol{a})) \\
&\sigma_1(\boldsymbol{x};\boldsymbol{b}), \ldots, \sigma_8(\boldsymbol{x};\boldsymbol{b}) \in \Sigma[\boldsymbol{x}]
\end{aligned}
$$

- When σ_5, σ_8 are known, **Linear Matrix Inequalities (LMIs)** $\Longrightarrow$ convex, solved by **Semidefinite Programming (SDP)**.
- In general, **Bilinear Matrix Inequalities (BMIs)** $\Longrightarrow$ non-convex and NP-hard.
- Similar for positive invariants.

SOS Constraints for Inductive Invariants

$$
\begin{aligned}
&\text{find} \quad \boldsymbol{a}, \boldsymbol{b} \\
-I(\boldsymbol{x};\boldsymbol{a}) + \epsilon &= \sigma_1(\boldsymbol{x};\boldsymbol{b}) + \sigma_2(\boldsymbol{x};\boldsymbol{b}) \cdot (-p(\boldsymbol{x})), \\
-I(f(\boldsymbol{x});\boldsymbol{a}) + \epsilon &= \sigma_3(\boldsymbol{x};\boldsymbol{b}) + \sigma_4(\boldsymbol{x};\boldsymbol{b}) \cdot (-g(\boldsymbol{x})) + \sigma_5(\boldsymbol{x};\boldsymbol{b}) \cdot (-I(\boldsymbol{x};\boldsymbol{a})) \\
-q(\boldsymbol{x}) + \epsilon &= \sigma_6(\boldsymbol{x};\boldsymbol{b}) + \sigma_7(\boldsymbol{x};\boldsymbol{b}) \cdot g(\boldsymbol{x}) + \sigma_8(\boldsymbol{x};\boldsymbol{b}) \cdot (-I(\boldsymbol{x};\boldsymbol{a})) \\
&\sigma_1(\boldsymbol{x};\boldsymbol{b}), \ldots, \sigma_8(\boldsymbol{x};\boldsymbol{b}) \in \Sigma[\boldsymbol{x}]
\end{aligned}
$$

- When σ_5, σ_8 are known, **Linear Matrix Inequalities (LMIs)** $\Longrightarrow$ convex, solved by **Semidefinite Programming (SDP)**.
- In general, **Bilinear Matrix Inequalities (BMIs)** $\Longrightarrow$ non-convex and NP-hard.
- Similar for positive invariants.

Challenges

Three challenges in optimization-based invariant generation:

1. How to simplify the (nonconvex) invariant conditions so that convex optimization methods (i.e. SDP) can be used?
   $\Longrightarrow$ **Template Setting**
2. How to encode the conditions over unbounded domains?
   $\Longrightarrow$ **Condition Encoding**
3. How to solve the BMI constraints efficiently?
   $\Longrightarrow$ **Constraint Solving**

Challenges

Three challenges in optimization-based invariant generation:

1. How to simplify the (nonconvex) invariant conditions so that convex optimization methods (i.e. SDP) can be used?
   $\Longrightarrow$ **Template Setting**
2. How to encode the conditions over unbounded domains?
   $\Longrightarrow$ **Condition Encoding**
3. How to solve the BMI constraints efficiently?
   $\Longrightarrow$ **Constraint Solving**

Challenges

Three challenges in optimization-based invariant generation:

1. How to simplify the (nonconvex) invariant conditions so that convex optimization methods (i.e. SDP) can be used?
   $\Longrightarrow$ **Template Setting**
2. How to encode the conditions over unbounded domains?
   $\Longrightarrow$ **Condition Encoding**
3. How to solve the BMI constraints efficiently?
   $\Longrightarrow$ **Constraint Solving**

Weak Invariants vs Strong Invariants

- Given an invariant template $I(\boldsymbol{x}; \boldsymbol{a})$.
- The **weak** invariant generation problem asks for one invariant satisfying the template, i.e., to find a valid parameter assignment.
- The **strong** invariant generation problem asks for a characterization of all possible invariants, i.e., to compute the set of valid parameter assignments.
  - Computing under-approximations of strong invariants can be reduced to convex optimization problems.

Weak Invariants vs Strong Invariants

- Given an invariant template $I(\boldsymbol{x}; \boldsymbol{a})$.
- The **weak** invariant generation problem asks for one invariant satisfying the template, i.e., to find a valid parameter assignment.
- The **strong** invariant generation problem asks for a characterization of all possible invariants, i.e., to compute the set of valid parameter assignments.
  - Computing under-approximations of strong invariants can be reduced to convex optimization problems.

Weak Invariants vs Strong Invariants

- Given an invariant template $I(\boldsymbol{x}; \boldsymbol{a})$.
- The **weak** invariant generation problem asks for one invariant satisfying the template, i.e., to find a valid parameter assignment.
- The **strong** invariant generation problem asks for a characterization of all possible invariants, i.e., to compute the set of valid parameter assignments.
  - Computing under-approximations of strong invariants can be reduced to convex optimization problems.

Dealing with BMI Constraints

- The quantified constraints:

$$\forall \mathbf{x}.\ \bigwedge_{i=1}^{m} k_i(\mathbf{x};\mathbf{a}) \geq 0 \implies l(\mathbf{x};\mathbf{a}) \geq 0$$

  where we assume bounded domain constraints $\boldsymbol{x} \in C_{\boldsymbol{x}}$ and $\boldsymbol{a} \in C_{\mathbf{a}}$.

- Question: to characterize the set of solutions for $\boldsymbol{a}$, denoted by R.
- Idea: to treat parameters as variables

$$\exists \mathbf{a}, \forall \mathbf{x}.\ \bigwedge_{i=1}^{m} k_i(\mathbf{a},\mathbf{x}) \geq 0 \implies l(\mathbf{a},\mathbf{x}) \geq 0$$

  then the problem becomes a **robust optimization** problem.

Dealing with BMI Constraints

- The quantified constraints:

$$\forall \mathbf{x}.\ \bigwedge_{i=1}^{m} k_i(\mathbf{x};\mathbf{a}) \geq 0 \implies l(\mathbf{x};\mathbf{a}) \geq 0$$

  where we assume bounded domain constraints $\boldsymbol{x} \in C_{\boldsymbol{x}}$ and $\boldsymbol{a} \in C_{\boldsymbol{a}}$.
- Question: to characterize the set of solutions for $\boldsymbol{a}$, denoted by R.
- Idea: to treat parameters as variables

$$\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \bigwedge_{i=1}^{m} k_i(\boldsymbol{a},\boldsymbol{x}) \geq 0 \implies l(\boldsymbol{a},\boldsymbol{x}) \geq 0$$

  then the problem becomes a **robust optimization** problem.

Dealing with BMI Constraints

- The quantified constraints:

$$\forall \mathbf{x}.\ \bigwedge_{i=1}^{m} k_i(\mathbf{x};\mathbf{a}) \geq 0 \implies l(\mathbf{x};\mathbf{a}) \geq 0$$

  where we assume bounded domain constraints $\boldsymbol{x} \in C_{\boldsymbol{x}}$ and $\boldsymbol{a} \in C_{\boldsymbol{a}}$.
- Question: to characterize the set of solutions for $\boldsymbol{a}$, denoted by R.
- Idea: to treat parameters as variables

$$\exists \mathbf{a}, \forall \mathbf{x}.\ \bigwedge_{i=1}^{m} k_i(\mathbf{a},\mathbf{x}) \geq 0 \implies l(\mathbf{a},\mathbf{x}) \geq 0$$

  then the problem becomes a **robust optimization** problem.

Representing Functions

- Consider expressing R as

$$R = \{\mathbf{a} \mid \phi(\mathbf{a}) \leq 0\}.$$

where ϕ can be proved to be upper-semicontinuous, i.e., can be approximated from above by polynomials.

- Under-approximations of set R can be obtained by approximating ϕ from above:

$$\forall \mathbf{a}.\ \phi(\mathbf{a}) \leq p_i(\mathbf{a}) \implies \underbrace{\{\mathbf{a} \mid p_i(\mathbf{a}) \leq 0\}}_{R_i} \subseteq \underbrace{\{\mathbf{a} \mid \phi(\mathbf{a}) \leq 0\}}_{R}$$

- Intuitively, the better p_i approximates ϕ, the less conservative R_i approximates R.

Representing Functions

- Consider expressing R as

$$R = \{\mathbf{a} \mid \phi(\mathbf{a}) \leq 0\}.$$

where ϕ can be proved to be upper-semicontinuous, i.e., can be approximated from above by polynomials.

- Under-approximations of set R can be obtained by approximating ϕ from above:

$$\forall \mathbf{a}.\ \phi(\mathbf{a}) \leq p_i(\mathbf{a}) \implies \underbrace{\{\mathbf{a} \mid p_i(\mathbf{a}) \leq 0\}}_{R_i} \subseteq \underbrace{\{\mathbf{a} \mid \phi(\mathbf{a}) \leq 0\}}_{R}$$

- Intuitively, the better p_i approximates ϕ, the less conservative R_i approximates R.

Representing Functions

- Consider expressing R as

$$R = \{\mathbf{a} \mid \phi(\mathbf{a}) \leq 0\}.$$

where ϕ can be proved to be upper-semicontinuous, i.e., can be approximated from above by polynomials.

- Under-approximations of set R can be obtained by approximating ϕ from above:

$$\forall \mathbf{a}.\ \phi(\mathbf{a}) \leq p_i(\mathbf{a}) \implies \underbrace{\{\mathbf{a} \mid p_i(\mathbf{a}) \leq 0\}}_{R_i} \subseteq \underbrace{\{\mathbf{a} \mid \phi(\mathbf{a}) \leq 0\}}_{R}$$

- Intuitively, the better p_i approximates ϕ, the less conservative R_i approximates R.

Computing p_i by SDP

- Set another template for p_i.

$$\begin{aligned} \min \quad & \int_{C_a} p_i(\mathbf{a}) \mathrm{d}\mu(\mathbf{a}) && \text{(minimize L1-norm over } C_{\mathbf{a}}\text{)} \\ s.t. \quad & p_i(\mathbf{a}) \geq l(\mathbf{a}, \mathbf{x}),\ \forall (\mathbf{a}, \mathbf{x}) \in C_a \times K_i(\mathbf{a}) \\ & \underbrace{p_i(\mathbf{a}) \geq -M,\ \forall (\mathbf{a}, \mathbf{x}) \in C_a \times C_{\mathbf{x}}}_{p_i(\mathbf{a}) \text{ approximates } \phi(\mathbf{a}) \text{ from above}}. \end{aligned}$$

- The under-approximations $R_i = \{\mathbf{a} \in C_{\mathbf{a}} \mid p_i(\mathbf{a}) \leq 0\}$ converge to R w.r.t. L_1 norm as $\deg(p) \to \infty$.

Theorem (Weak Completeness)

If $\partial R = \{a \mid \phi(a) = 0\}$ has measure zero and the valid set R has positive measure, our algorithm can find an non-empty under-approximation.

Computing p_i by SDP

- Set another template for p_i.

$$\begin{aligned} \min \quad & \int_{C_a} p_i(\mathbf{a}) \mathrm{d}\mu(\mathbf{a}) && \text{(minimize L1-norm over } C_{\mathbf{a}}\text{)} \\ s.t. \quad & p_i(\mathbf{a}) \geq l(\mathbf{a}, \mathbf{x}),\ \forall (\mathbf{a}, \mathbf{x}) \in C_a \times K_i(\mathbf{a}) \\ & \underbrace{p_i(\mathbf{a}) \geq -M,\ \forall (\mathbf{a}, \mathbf{x}) \in C_a \times C_{\mathbf{x}}}_{p_i(\mathbf{a}) \text{ approximates } \phi(\mathbf{a}) \text{ from above}}. \end{aligned}$$

- The under-approximations $R_i = \{\mathbf{a} \in C_{\mathbf{a}} \mid p_i(\mathbf{a}) \leq 0\}$ converge to R w.r.t. L_1 norm as $\deg(p) \to \infty$.

Theorem (Weak Completeness)

If $\partial R = \{a \mid \phi(a) = 0\}$ has measure zero and the valid set R has positive measure, our algorithm can find an non-empty under-approximation.

Pros, cons, and extensions

- **Pros**
  - Allow the template $I(\boldsymbol{x}; \boldsymbol{a})$ to be nonlinear in $\boldsymbol{a}$, or even a parameterized formula.
  - Soundness, convergence, and weak completeness guarantee.
- **Cons**
  - SDP constraint size is polynomial in $|\boldsymbol{a}| + |\boldsymbol{x}|$.
  - Weak completeness requires the assumption that R has positive-measure.
- When R has zero-measure, for a special subclass of templates, we give a sound and complete SDP-based algorithm by using variable substitution:
  - SDP constraint size is polynomial only in $|\boldsymbol{x}|$.
  - Shown to be much more efficient than other constraint-based methods.

Pros, cons, and extensions

- **Pros**
  - Allow the template $I(\boldsymbol{x}; \boldsymbol{a})$ to be nonlinear in $\boldsymbol{a}$, or even a parameterized formula.
  - Soundness, convergence, and weak completeness guarantee.
- **Cons**
  - SDP constraint size is polynomial in $|\boldsymbol{a}| + |\boldsymbol{x}|$.
  - Weak completeness requires the assumption that R has positive-measure.
- When R has zero-measure, for a special subclass of templates, we give a sound and complete SDP-based algorithm by using variable substitution:
  - SDP constraint size is polynomial only in $|\boldsymbol{x}|$.
  - Shown to be much more efficient than other constraint-based methods.

Pros, cons, and extensions

- **Pros**
  - Allow the template $I(\boldsymbol{x}; \boldsymbol{a})$ to be nonlinear in $\boldsymbol{a}$, or even a parameterized formula.
  - Soundness, convergence, and weak completeness guarantee.
- **Cons**
  - SDP constraint size is polynomial in $|\boldsymbol{a}| + |\boldsymbol{x}|$.
  - Weak completeness requires the assumption that R has positive-measure.
- When R has zero-measure, for a special subclass of templates, we give a sound and complete SDP-based algorithm by using variable substitution:
  - SDP constraint size is polynomial only in $|\boldsymbol{x}|$.
  - Shown to be much more efficient than other constraint-based methods.

Problem Formulation

- For systems with unbounded domains, encoding SOS constraints is sound but not relatively complete
  - Putinar's Psatz is not applicable, due to the violation of the Archimedean condition.
- Question: How to retain completeness?
- Idea: project the unbounded domain into a bounded region.

Problem Formulation

- For systems with unbounded domains, encoding SOS constraints is sound but not relatively complete
  - Putinar's Psatz is not applicable, due to the violation of the Archimedean condition.
- Question: How to retain completeness?
- Idea: project the unbounded domain into a bounded region.

Problem Formulation

- For systems with unbounded domains, encoding SOS constraints is sound but not relatively complete
  - Putinar's Psatz is not applicable, due to the violation of the Archimedean condition.
- Question: How to retain completeness?
- Idea: project the unbounded domain into a bounded region.

Problem Formulation

- For systems with unbounded domains, encoding SOS constraints is sound but not relatively complete
  - Putinar's Psatz is not applicable, due to the violation of the Archimedean condition.
- Question: How to retain completeness?
- Idea: project the unbounded domain into a bounded region.

Algebraic Tool: Homogenization

- Introduce a fresh variable x_0. Denote $\tilde{\mathbf{x}} = (x_0, \mathbf{x}) \in \mathbb{R}^{n+1}$.
- Let $p(\mathbf{x})$ be a polynomial of degree d, its **homogenization** is

$$\tilde{p}(\tilde{\mathbf{x}}) \triangleq x_0^d \cdot p(\frac{x_1}{x_0}, \ldots, \frac{x_n}{x_0}).$$

$$p(x_1, x_2) = x_1^2 + x_2 + 1 \implies \tilde{p}(x_0, x_1, x_2) = x_1^2 + x_2 x_0 + x_0^2.$$

- Let $S = \{\mathbf{x} \in \mathbb{R}^n \mid p_1(\mathbf{x}) \geq 0, \ldots, p_m(\mathbf{x}) \geq 0\}$, define

$$\tilde{S} \triangleq \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \ldots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, \underbrace{x_0 \geq 0, \|\tilde{\mathbf{x}}\|^2 = 1}_{\text{half unit sphere in } \mathbb{R}^{n+1}}\}.$$

Algebraic Tool: Homogenization

- Introduce a fresh variable x_0. Denote $\tilde{\mathbf{x}} = (x_0, \mathbf{x}) \in \mathbb{R}^{n+1}$.
- Let $p(\mathbf{x})$ be a polynomial of degree d, its **homogenization** is

$$\tilde{p}(\tilde{\mathbf{x}}) \doteq x_0^d \cdot p(\frac{x_1}{x_0}, \ldots, \frac{x_n}{x_0}).$$

$$p(x_1, x_2) = x_1^2 + x_2 + 1 \implies \tilde{p}(x_0, x_1, x_2) = x_1^2 + x_2 x_0 + x_0^2.$$

- Let $S = \{\mathbf{x} \in \mathbb{R}^n \mid p_1(\mathbf{x}) \geq 0, \ldots, p_m(\mathbf{x}) \geq 0\}$, define

$$\tilde{S} \doteq \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \ldots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, \underbrace{x_0 \geq 0, \|\tilde{\mathbf{x}}\|^2 = 1}_{\text{half unit sphere in } \mathbb{R}^{n+1}}\}.$$

Algebraic Tool: Homogenization

- Introduce a fresh variable x_0. Denote $\tilde{\mathbf{x}} = (x_0, \mathbf{x}) \in \mathbb{R}^{n+1}$.
- Let $p(\mathbf{x})$ be a polynomial of degree d, its **homogenization** is

$$\tilde{p}(\tilde{\mathbf{x}}) \doteq x_0^d \cdot p(\frac{x_1}{x_0}, \ldots, \frac{x_n}{x_0}).$$

$$p(x_1, x_2) = x_1^2 + x_2 + 1 \implies \tilde{p}(x_0, x_1, x_2) = x_1^2 + x_2 x_0 + x_0^2.$$

- Let $S = \{\mathbf{x} \in \mathbb{R}^n \mid p_1(\mathbf{x}) \geq 0, \ldots, p_m(\mathbf{x}) \geq 0\}$, define

$$\tilde{S} \doteq \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \ldots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, \underbrace{x_0 \geq 0, \|\tilde{\mathbf{x}}\|^2 = 1}_{\text{half unit sphere in } \mathbb{R}^{n+1}}\}.$$

Relation between S and $\tilde{S}$

$$S = \{\mathbf{x} \in \mathbb{R}^n \mid p_1(\mathbf{x}) \geq 0, \ldots, p_m(\mathbf{x}) \geq 0\},$$
$$\tilde{S} = \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \ldots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, x_0 \geq 0, \|\tilde{\mathbf{x}}\|^2 = 1\},$$
$$\tilde{S}_+ = \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \ldots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, x_0 > 0, \|\tilde{\mathbf{x}}\|^2 = 1\}.$$

- "$S = \tilde{S}_+$": an one-to-one projection

$$\mathbf{x} \in S \iff \left(\frac{1}{\sqrt{1 + \|\mathbf{x}\|^2}}, \frac{x_1}{\sqrt{1 + \|\mathbf{x}\|^2}}, \ldots, \frac{x_n}{\sqrt{1 + \|\mathbf{x}\|^2}} \right) \in \tilde{S}_+.$$

- closure$(\tilde{S}_+) = \tilde{S}$

Relation between S and $\tilde{S}$

$$S = \{\mathbf{x} \in \mathbb{R}^n \mid p_1(\mathbf{x}) \geq 0, \ldots, p_m(\mathbf{x}) \geq 0\},$$
$$\tilde{S} = \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \ldots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, x_0 \geq 0, \|\tilde{\mathbf{x}}\|^2 = 1\},$$
$$\tilde{S}_+ = \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \ldots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, x_0 > 0, \|\tilde{\mathbf{x}}\|^2 = 1\}.$$

- "$S = \tilde{S}_+$": an one-to-one projection

$$\mathbf{x} \in S \iff \left(\frac{1}{\sqrt{1+\|\mathbf{x}\|^2}}, \frac{x_1}{\sqrt{1+\|\mathbf{x}\|^2}}, \ldots, \frac{x_n}{\sqrt{1+\|\mathbf{x}\|^2}}\right) \in \tilde{S}_+.$$

- closure$(\tilde{S}_+) = \tilde{S}$

Relation between S and $\tilde{S}$

$$
\begin{aligned}
S &= \{\mathbf{x} \in \mathbb{R}^n \mid p_1(\mathbf{x}) \geq 0, \ldots, p_m(\mathbf{x}) \geq 0\},\\
\tilde{S} &= \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \ldots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, x_0 \geq 0, \|\tilde{\mathbf{x}}\|^2 = 1\},\\
\tilde{S}_+ &= \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \ldots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, x_0 > 0, \|\tilde{\mathbf{x}}\|^2 = 1\}.
\end{aligned}
$$

- "$S = \tilde{S}_+$": an one-to-one projection

$$
\mathbf{x} \in S \iff \left(\frac{1}{\sqrt{1+\|\mathbf{x}\|^2}}, \frac{x_1}{\sqrt{1+\|\mathbf{x}\|^2}}, \ldots, \frac{x_n}{\sqrt{1+\|\mathbf{x}\|^2}} \right) \in \tilde{S}_+.
$$

- closure$(\tilde{S}_+) = \tilde{S}$

Relation between S and $\tilde{S}$

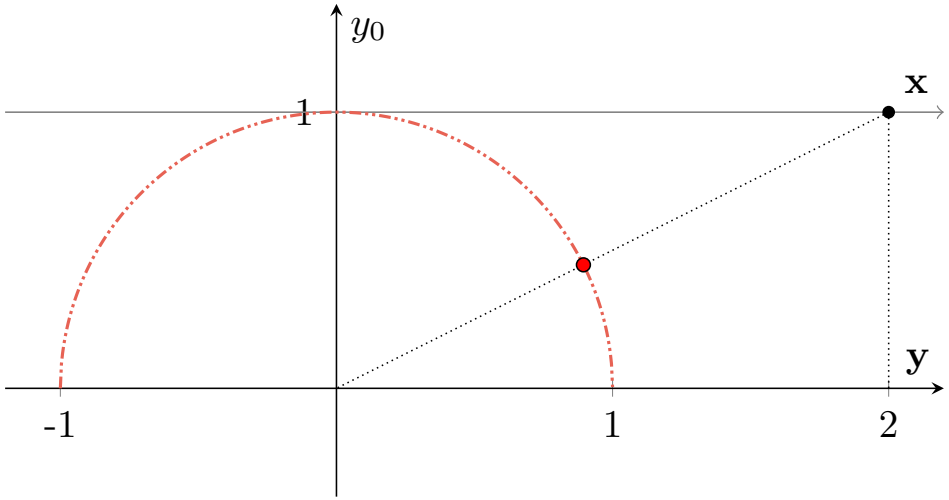
$$
\begin{aligned}
S &= \{\mathbf{x} \in \mathbb{R}^n \mid p_1(\mathbf{x}) \geq 0, \ldots, p_m(\mathbf{x}) \geq 0\},\\
\tilde{S} &= \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \ldots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, x_0 \geq 0, \|\tilde{\mathbf{x}}\|^2 = 1\},\\
\tilde{S}_+ &= \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \ldots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, x_0 > 0, \|\tilde{\mathbf{x}}\|^2 = 1\}.
\end{aligned}
$$

- "$S = \tilde{S}_+$": an one-to-one projection

$$
\mathbf{x} \in S \iff \left(\frac{1}{\sqrt{1+\|\mathbf{x}\|^2}}, \frac{x_1}{\sqrt{1+\|\mathbf{x}\|^2}}, \ldots, \frac{x_n}{\sqrt{1+\|\mathbf{x}\|^2}} \right) \in \tilde{S}_+.
$$

- closure$(\tilde{S}_+) = \tilde{S}$

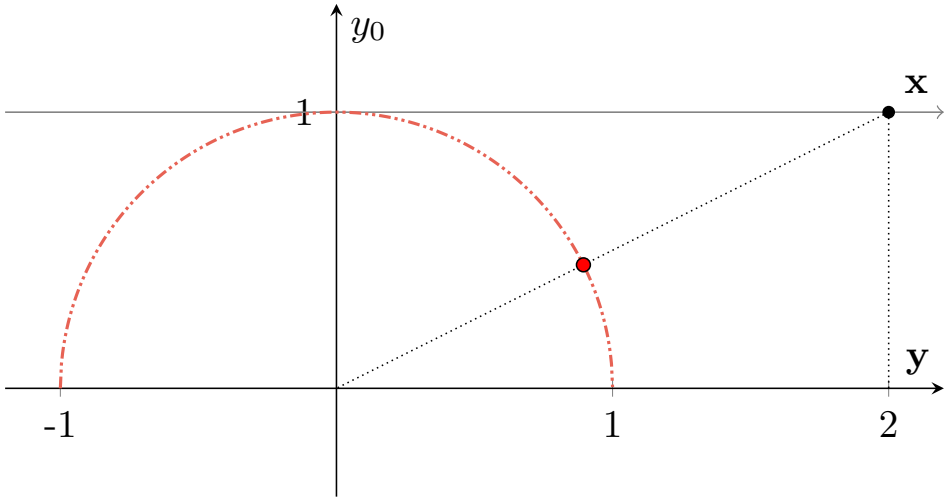
Geometric Explanation for $\mathbb{R}^1$



$$x \in \mathbb{R} \longleftrightarrow (y_0, y) \hat{=} (\underbrace{\frac{1}{\sqrt{1+x^2}}}_{x_0}, \underbrace{\frac{x}{\sqrt{1+x^2}}}_{\text{new } x}) \in \tilde{S}_+$$

- Points with $x_0 = 0$ in $\tilde{S}$ correspond to **points at infinity** in $\mathbb{R}^n$.
- We need a technical assumption for these points.

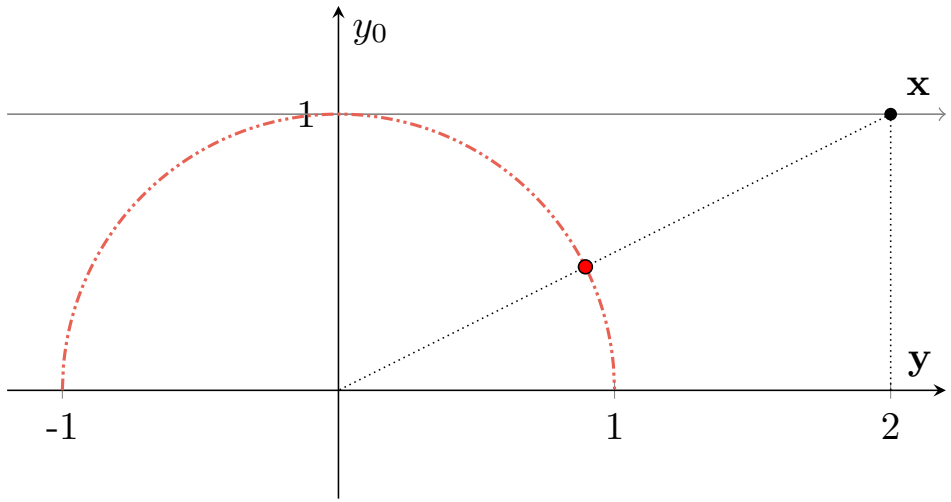
Geometric Explanation for $\mathbb{R}^1$



$$x \in \mathbb{R} \longleftrightarrow (y_0, y) \hat{=} (\underbrace{\frac{1}{\sqrt{1+x^2}}}_{x_0}, \underbrace{\frac{x}{\sqrt{1+x^2}}}_{\text{new } x}) \in \tilde{S}_+$$

- Points with $x_0 = 0$ in $\tilde{S}$ correspond to **points at infinity** in $\mathbb{R}^n$.
- We need a technical assumption for these points.

Geometric Explanation for $\mathbb{R}^1$



$$x \in \mathbb{R} \longleftrightarrow (y_0, y) \hat{=} (\underbrace{\frac{1}{\sqrt{1+x^2}}}_{x_0}, \underbrace{\frac{x}{\sqrt{1+x^2}}}_{\text{new } x}) \in \tilde{S}_+$$

- Points with $x_0 = 0$ in $\tilde{S}$ correspond to **points at infinity** in $\mathbb{R}^n$.
- We need a technical assumption for these points.

Closed at infinity

Definition (closed at infinity [Nie, 2012])

A basic semialgebraic set S is closed at ∞ if

$$\text{closure}(\tilde{S}_+) = \tilde{S}.$$

- Depends on the description

$$S_1 = \{(x_1, x_2) \in \mathbb{R}^2 \mid x_1 - x_2^2 \geq 0\} \text{ not closed at } \infty$$

$$S_2 = \{(x_1, x_2) \in \mathbb{R}^2 \mid x_1 - x_2^2 \geq 0, \underbrace{x_1 \geq 0}_{\text{redundant}}\} \text{ closed at } \infty$$

Homogenization: Main Theorem

Theorem (Equivalence [Huang et al., 2023])

When S is closed at ∞,

$$f(\mathbf{x}) \geq 0 \text{ over } S \iff \tilde{f}(\tilde{\mathbf{x}}) \geq 0 \text{ over } \tilde{S}$$

- $\tilde{S}$ lies on the half unit sphere, hence bounded.
- The quadratic module $\mathbf{QM}(\tilde{S})$ becomes Archimedean.

Applied for (differential) invariants

- Original SOS constraints (only sound), for any $\epsilon > 0$:

$$
\begin{aligned}
&-B(\mathbf{x}) \in \mathbf{QM}(\mathcal{X}_i),\\
&B(\mathbf{x}) - \epsilon \in \mathbf{QM}(\mathcal{X}_u)\\
&\lambda(\mathbf{x})B(\mathbf{x}) - \mathfrak{L}_{\boldsymbol{f}}B(\mathbf{x}) \in \mathbf{QM}(\mathcal{X}_d)
\end{aligned}
$$

- First **homogenization**, then **Putinar's Psatz**.

$$
\begin{aligned}
&-\tilde{B}(x_0,\mathbf{x}) \in \mathbf{QM}(\tilde{\mathcal{X}}_i)\\
&\tilde{B}(x_0,\mathbf{x}) - \epsilon x_0^d \in \mathbf{QM}(\tilde{\mathcal{X}}_u)\\
&\tilde{H}(x_0,\mathbf{x}) \in \mathbf{QM}(\tilde{\mathcal{X}}_d)
\end{aligned}
$$

  where $H(\mathbf{x}) = \lambda(\mathbf{x})B(\mathbf{x}) - \mathfrak{L}_{\boldsymbol{f}}B(\mathbf{x})$.

- Assume that $\mathcal{X}_i$, $\mathcal{X}_u$, and $\mathcal{X}_d$ are all closed at infinity. Under some mild conditions (ϵ can take 0), both sound and relatively complete.

Applied for (differential) invariants

- Original SOS constraints (only sound), for any $\epsilon > 0$:

$$
\begin{aligned}
&-B(\mathbf{x}) \in \mathbf{QM}(\mathcal{X}_i),\\
&B(\mathbf{x}) - \epsilon \in \mathbf{QM}(\mathcal{X}_u)\\
&\lambda(\mathbf{x})B(\mathbf{x}) - \mathfrak{L}_{\boldsymbol{f}}B(\mathbf{x}) \in \mathbf{QM}(\mathcal{X}_d)
\end{aligned}
$$

- First **homogenization**, then **Putinar's Psatz**.

$$
\begin{aligned}
&-\tilde{B}(x_0, \mathbf{x}) \in \mathbf{QM}(\tilde{\mathcal{X}}_i)\\
&\tilde{B}(x_0, \mathbf{x}) - \epsilon x_0^d \in \mathbf{QM}(\tilde{\mathcal{X}}_u)\\
&\tilde{H}(x_0, \mathbf{x}) \in \mathbf{QM}(\tilde{\mathcal{X}}_d)
\end{aligned}
$$

  where $H(\mathbf{x}) = \lambda(\mathbf{x})B(\mathbf{x}) - \mathfrak{L}_{\boldsymbol{f}}B(\mathbf{x})$.

- Assume that $\mathcal{X}_i$, $\mathcal{X}_u$, and $\mathcal{X}_d$ are all closed at infinity. Under some mild conditions (ϵ can take 0), both sound and relatively complete.

Applied for (differential) invariants

- Original SOS constraints (only sound), for any $\epsilon > 0$:

$$
\begin{aligned}
&-B(\mathbf{x}) \in \mathbf{QM}(\mathcal{X}_i),\\
&B(\mathbf{x}) - \epsilon \in \mathbf{QM}(\mathcal{X}_u)\\
&\lambda(\mathbf{x})B(\mathbf{x}) - \mathfrak{L}_{\boldsymbol{f}}B(\mathbf{x}) \in \mathbf{QM}(\mathcal{X}_d)
\end{aligned}
$$

- First **homogenization**, then **Putinar's Psatz**.

$$
\begin{aligned}
&-\tilde{B}(x_0, \mathbf{x}) \in \mathbf{QM}(\tilde{\mathcal{X}}_i)\\
&\tilde{B}(x_0, \mathbf{x}) - \epsilon x_0^d \in \mathbf{QM}(\tilde{\mathcal{X}}_u)\\
&\tilde{H}(x_0, \mathbf{x}) \in \mathbf{QM}(\tilde{\mathcal{X}}_d)
\end{aligned}
$$

  where $H(\mathbf{x}) = \lambda(\mathbf{x})B(\mathbf{x}) - \mathfrak{L}_{\boldsymbol{f}}B(\mathbf{x})$.

- Assume that $\mathcal{X}_i$, $\mathcal{X}_u$, and $\mathcal{X}_d$ are all closed at infinity. Under some mild conditions (ϵ can take 0), both sound and relatively complete.

Semialgebraic Barrier Certificates

- Set a template for $B(\boldsymbol{x}) \in \mathbb{R}[\boldsymbol{x}]$, then compute $\tilde{B}(x_0, \boldsymbol{x})$.
- Alternatively, we can set a polynomial template for $\tilde{B}(x_0, \boldsymbol{x})$, then the original $B(\frac{1}{\sqrt{\|\boldsymbol{x}\|^2+1}}, \frac{\boldsymbol{x}}{\sqrt{\|\boldsymbol{x}\|^2+1}})$ can be non-polynomial:

$$(\sqrt{\|\boldsymbol{x}\|^2+1})^d \cdot B(\frac{1}{\sqrt{\|\boldsymbol{x}\|^2+1}}, \frac{\boldsymbol{x}}{\sqrt{\|\boldsymbol{x}\|^2+1}})$$
$$= \underbrace{B_1(\boldsymbol{x}) + \sqrt{\|\boldsymbol{x}\|^2+1} \cdot B_2(\boldsymbol{x})}_{\text{semialgebraic BC}} \quad B_1, B_2 \in \mathbb{R}[\boldsymbol{x}]$$

- Semialgebraic BC subsumes polynomial BC as a special case, where $B_2(\boldsymbol{x})$ is constant zero.

Semialgebraic Barrier Certificates

- Set a template for $B(\boldsymbol{x}) \in \mathbb{R}[\boldsymbol{x}]$, then compute $\tilde{B}(x_0, \boldsymbol{x})$.
- Alternatively, we can set a polynomial template for $\tilde{B}(x_0, \boldsymbol{x})$, then the original $B(\frac{1}{\sqrt{\|\boldsymbol{x}\|^2+1}}, \frac{\boldsymbol{x}}{\sqrt{\|\boldsymbol{x}\|^2+1}})$ can be non-polynomial:

$$
\begin{aligned}
&(\sqrt{\|\mathbf{x}\|^2+1})^d \cdot B(\frac{1}{\sqrt{\|\mathbf{x}\|^2+1}}, \frac{\mathbf{x}}{\sqrt{\|\mathbf{x}\|^2+1}}) \\
&= \underbrace{B_1(\mathbf{x}) + \sqrt{\|\mathbf{x}\|^2+1} \cdot B_2(\mathbf{x})}_{\textbf{semialgebraic BC}} \quad B_1, B_2 \in \mathbb{R}[\mathbf{x}]
\end{aligned}
$$

- Semialgebraic BC subsumes polynomial BC as a special case, where $B_2(\boldsymbol{x})$ is constant zero.

Semialgebraic Barrier Certificates

- Set a template for $B(\boldsymbol{x}) \in \mathbb{R}[\boldsymbol{x}]$, then compute $\tilde{B}(x_0, \boldsymbol{x})$.
- Alternatively, we can set a polynomial template for $\tilde{B}(x_0, \boldsymbol{x})$, then the original $B(\frac{1}{\sqrt{\|\boldsymbol{x}\|^2+1}}, \frac{\boldsymbol{x}}{\sqrt{\|\boldsymbol{x}\|^2+1}})$ can be non-polynomial:

$$
\begin{aligned}
&(\sqrt{\|\mathbf{x}\|^2+1})^d \cdot B(\frac{1}{\sqrt{\|\mathbf{x}\|^2+1}}, \frac{\mathbf{x}}{\sqrt{\|\mathbf{x}\|^2+1}}) \\
&= \underbrace{B_1(\mathbf{x}) + \sqrt{\|\mathbf{x}\|^2+1} \cdot B_2(\mathbf{x})}_{\textbf{semialgebraic BC}} \quad B_1, B_2 \in \mathbb{R}[\mathbf{x}]
\end{aligned}
$$

- Semialgebraic BC subsumes polynomial BC as a special case, where $B_2(\boldsymbol{x})$ is constant zero.

Experiments

			Incomplete			Polynomial BC			Semialgebraic BC		
system	dim	unbounded	deg	succ	time(s)	deg	succ	time(s)	deg	succ	time(s)
vector[37]	2	$\mathcal{X}$	4	✓	0.58	3	✓	0.03	**2**	✓	0.83
		$\mathcal{I},\mathcal{U},\mathcal{X}$	>6	✗	0.39	4	✓	0.16	**2**	✓	0.72
barrier[29]	2	$\mathcal{X}$	>6	✗	2.40	>6	✗	2.73	>4	✗	22.98
		$\mathcal{I},\mathcal{U},\mathcal{X}$	>6	✗	0.78	3	✓	0.04	**2**	✓	3.12
lie-der[24]	2	$\mathcal{X}$	3	✓	0.19	3	✓	0.14	**1**	✓	0.75
		$\mathcal{I},\mathcal{U},\mathcal{X}$	**3**	✓	0.14	**3**	✓	0.19	**3**	✓	5.04
arch1[36]	2	$\mathcal{X}$	**4**	✓	0.40	**4**	✓	0.54	>4	✗	117.61
		$\mathcal{I},\mathcal{U},\mathcal{X}$	**1**	✓	0.09	**1**	✓	0.04	2	✓	20.92
arch2[36]	2	$\mathcal{X}$	**3**	✓	0.20	**3**	✓	0.21	**3**	✓	5.53
		$\mathcal{I},\mathcal{U},\mathcal{X}$	3	✓	0.18	3	✓	0.18	**2**	✓	1.59
arch3[36]	2	$\mathcal{X}$	**2**	✓	0.12	**2**	✓	0.13	**2**	✓	3.45
		$\mathcal{I},\mathcal{U},\mathcal{X}$	>6	✗	0.84	>6	✗	1.30	**1**	✓	0.34
arch4[36]	2	$\mathcal{X}$	>6	✗	0.11	5	✓	0.74	**3**	✓	5.69
		$\mathcal{U},\mathcal{X}$	>6	✗	1.02	**6**	✓	1.21	>4	✗	7.74
nagumo[34]	2	$\mathcal{X}$	**2**	✓	0.13	**2**	✓	0.14	**2**	✓	3.50
		$\mathcal{U},\mathcal{X}$	>6	✗	1.02	**3**	✓	0.17	>4	✗	23.73
lorenz[10]	3	$\mathcal{X}$	6	?	TO	**4**	✓	72.13	2	?	TO
		$\mathcal{U},\mathcal{X}$	**5**	✓	248.88	6	?	TO	2	?	TO
lotka[14]	3	$\mathcal{X}$	>6	✗	88.97	>6	✗	27.8	3	?	TO
		$\mathcal{U},\mathcal{X}$	>6	✗	0.27	>6	✗	438.54	3	?	TO

- Expressiveness: Semialgebraic $>$ Polynomial $>$ Incomplete
- Efficiency: Incomplete $\approx$ Polynomial $\gg$ Semialgebraic

Solving the Positive Invariant Constraints

$$\forall \boldsymbol{x} \in \mathcal{X}_d.\ B(\boldsymbol{x}) = 0 \implies \mathfrak{L}_{\boldsymbol{f}} B(\boldsymbol{x}) < 0.$$

$$\lambda(\boldsymbol{x}; \boldsymbol{a}) B(\boldsymbol{x}; \boldsymbol{b}) - \mathfrak{L}_{\boldsymbol{f}} B(\boldsymbol{x}; \boldsymbol{b}) \in \mathbf{QM}(\mathcal{X}_d)$$

- When $\lambda(\boldsymbol{x})$ is known, the constraints can be reduced to **LMI**
  - fix $\lambda(\boldsymbol{x})$, solve as SDP. [Kong et al., 2013; Dai et al., 2017]
- In general, **BMI** constraints:
  - [Yang et al., 2015; Chen et al., 2020] alternating descent: efficient, no convergence guarantee
  - [Wang et al., 2021; Wang et al., 2022] **difference-of-convex (DC)**: guarantees to terminate and converge but requires good initial points

Solving the Positive Invariant Constraints

$$\forall \mathbf{x} \in \mathcal{X}_d.\ B(\mathbf{x}) = 0 \implies \mathfrak{L}_{\boldsymbol{f}} B(\mathbf{x}) < 0.$$

$$\lambda(\mathbf{x}; \boldsymbol{a}) B(\mathbf{x}; \boldsymbol{b}) - \mathfrak{L}_{\boldsymbol{f}} B(\mathbf{x}; \boldsymbol{b}) \in \mathbf{QM}(\mathcal{X}_d)$$

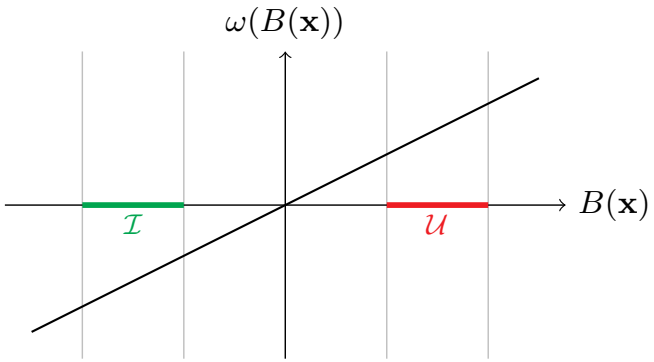
- When $\lambda(\boldsymbol{x})$ is known, the constraints can be reduced to **LMI**
  - fix $\lambda(\boldsymbol{x})$, solve as SDP. [Kong et al., 2013; Dai et al., 2017]
- In general, **BMI** constraints:
  - [Yang et al., 2015; Chen et al., 2020] alternating descent: efficient, no convergence guarantee
  - [Wang et al., 2021; Wang et al., 2022] **difference-of-convex (DC)**: guarantees to terminate and converge but requires good initial points

A Closer Look at $\lambda(\boldsymbol{x}; \boldsymbol{a})$

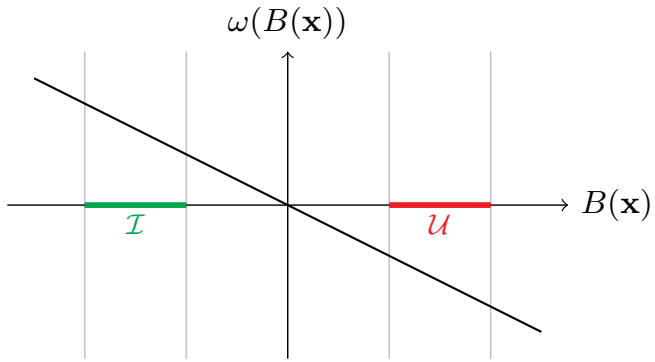
- Idea: choose a "good" $\lambda(\boldsymbol{x}; \boldsymbol{a})$ to attain expressiveness and to simplify the DC procedure.
- First, exponential-type conditions ($\lambda(\boldsymbol{x}; \boldsymbol{a})$ degenerates to a constant) are usually too conservative.

A Closer Look at $\lambda(\boldsymbol{x};\boldsymbol{a})$

- Idea: choose a "good" $\lambda(\boldsymbol{x};\boldsymbol{a})$ to attain expressiveness and to simplify the DC procedure.
- First, exponential-type conditions ($\lambda(\boldsymbol{x};\boldsymbol{a})$ degenerates to a constant) are usually too conservative.



(a) $\omega(b) = \lambda b,\ \lambda > 0$

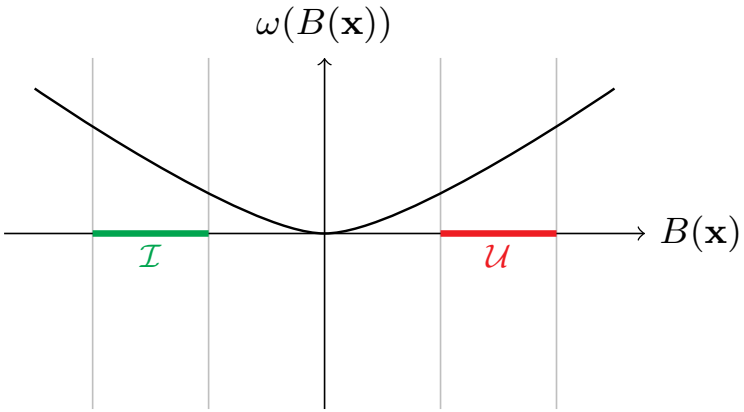


(b) $\omega(b) = \lambda b,\ \lambda < 0$

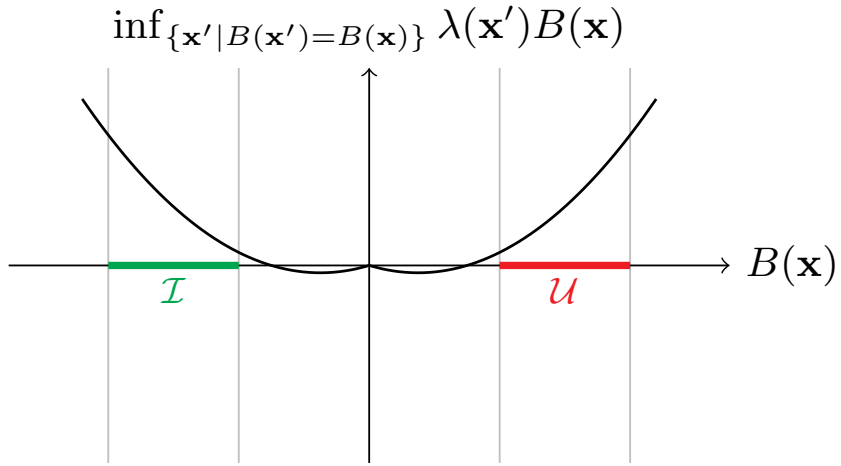
- Left: $\lambda > 0$, $\mathfrak{L}_{\boldsymbol{f}} B(\boldsymbol{x}) < 0$ whenever $B(\boldsymbol{x}) < 0$
  $\Longrightarrow$ trajectories must stabilize to the minimizer of $B(\boldsymbol{x})$.
- Right: $\lambda < 0$, $\mathfrak{L}_{\boldsymbol{f}} B(\boldsymbol{x}) < 0$ whenever $B(\boldsymbol{x}) > 0$
  $\Longrightarrow$ trajectories must stabilize to the region $B(\boldsymbol{x}) \leq 0$.

A Closer Look at $\lambda(\boldsymbol{x};\boldsymbol{a})$

- Ideally, we would like to choose $\lambda(\boldsymbol{x};\boldsymbol{a})$ to make $\lambda(\boldsymbol{x})B(\boldsymbol{x})$, denoted $\omega(B(\mathbf{x}))$, positive whenever $B(\mathbf{x}) \neq 0$.



(a) the ideal case for ω

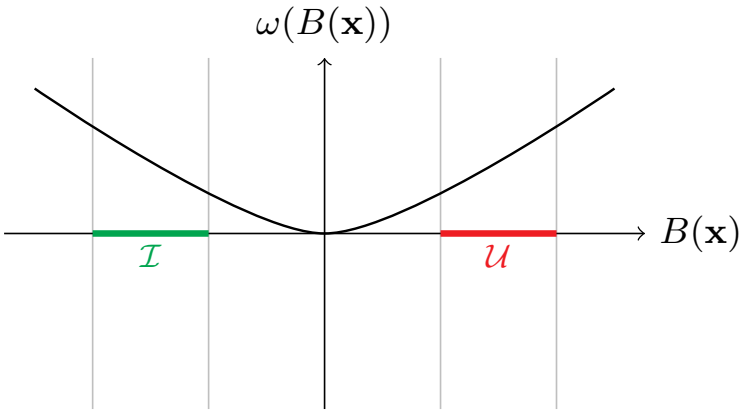


(b) $\lambda(\mathbf{x}) < 0$ over $\mathcal{X}_i$ and > 0 over $\mathcal{X}_u$

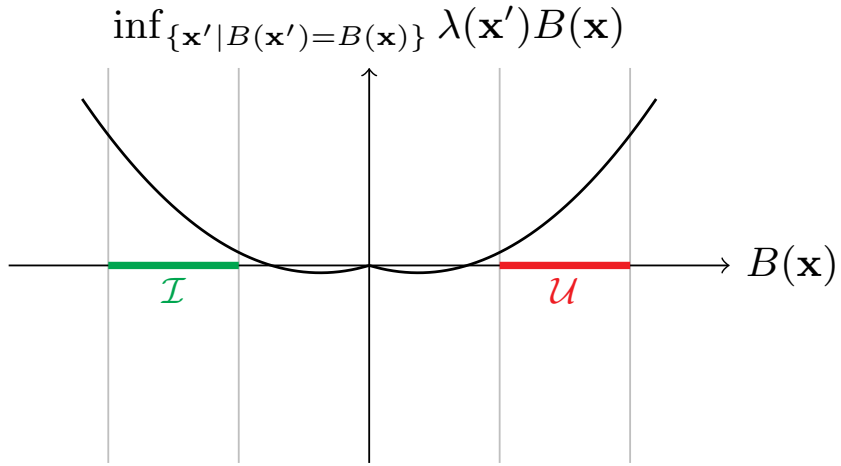
- Observation: we can approximate the ideal case by choosing a $\lambda(\boldsymbol{x};\boldsymbol{a}) < 0$ over $\mathcal{X}_i$ and > 0 over $\mathcal{X}_u$.
  - Such $\lambda(\boldsymbol{x})$ is called an **interpolant** between $\mathcal{X}_i$ and $\mathcal{X}_u$.
  - Find some "good" interpolants to serve as DC initial points.

A Closer Look at $\lambda(\boldsymbol{x};\boldsymbol{a})$

- Ideally, we would like to choose $\lambda(\boldsymbol{x};\boldsymbol{a})$ to make $\lambda(\boldsymbol{x})B(\boldsymbol{x})$, denoted $\omega(B(\boldsymbol{x}))$, positive whenever $B(\boldsymbol{x}) \neq 0$.



(a) the ideal case for ω



(b) $\lambda(\boldsymbol{x}) < 0$ over $\mathcal{X}_i$ and > 0 over $\mathcal{X}_u$

- Observation: we can approximate the ideal case by choosing a $\lambda(\boldsymbol{x};\boldsymbol{a}) < 0$ over $\mathcal{X}_i$ and > 0 over $\mathcal{X}_u$.
  - Such $\lambda(\boldsymbol{x})$ is called an **interpolant** between $\mathcal{X}_i$ and $\mathcal{X}_u$.
  - Find some "good" interpolants to serve as DC initial points.

A Simple Case Study

- The algorithm has been implemented in our toolchain **MARS 2.0**.
- A simple example

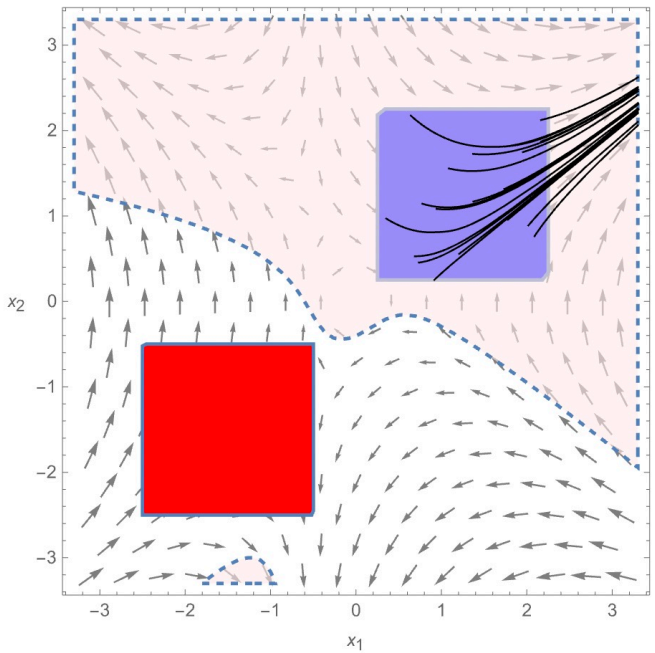
$$\begin{pmatrix} \dot{x}_1 \\ \dot{x}_2 \end{pmatrix} = \begin{pmatrix} x_2 + 2x_1x_2 \\ -x_1 - x_2^2 + 2x_1^2 \end{pmatrix}$$

with $\mathcal{X}_d = [-3, 3] \times [-3, 3]$, $\mathcal{X}_i = [0.25, 2.25] \times [0.25, 2.25]$, and $\mathcal{X}_u = [-2.5, -0.5] \times [-2.5, -0.5]$.

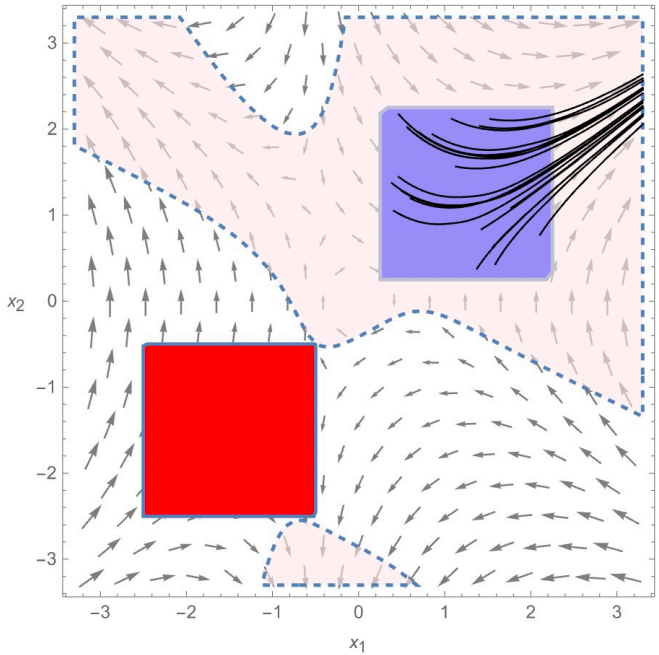
- Using SDP to solve the exponential-type BC condition [Kong et al., 2013] or using alternating descent method to solve non-convex BC condition [Yang et al., 2015] will fail to find a BC of degree $1 \sim 6$.

A Simple Case Study

- Interpolation&DC: find a BC at degree 4, taking 26.4s.
- Original DC: find a BC at degree 5, taking 291.5s.



(a) ItpDC: degree 4



(b) DC: degree 5

Summary

We show how to exploit advanced optimization techniques in the invariant generation problem for both discrete- and continuous-time dynamics:

- **Robust Optimization** techniques for under-approximating strong invariants.
- **Homogenization** techniques for encoding conditions over unbounded domains.
- **Non-Convex Optimization** techniques for solving constraint.

Future Direction: use sparsity to conquer scalability

- Reasoning about nonlinear arithmetic + quantifier alternation is inherently hard.
  - It is unrealistic to expect to find an general and efficient solution.
- In practice, we have these observations:
  - Nonlinearity usually appears locally among some variables.
  - Real invariants (especially for programs) usually contain very few monomials.
- Question: How to exploit such sparsity information in template setting and constraint solving?
- Our recent attempt: combine **treewidth** techniques (sparsity from graph theory) with **quantifier elimination**. $\Longrightarrow$ TACAS'26.

Future Direction: use sparsity to conquer scalability

- Reasoning about nonlinear arithmetic + quantifier alternation is inherently hard.
  - It is unrealistic to expect to find an general and efficient solution.
- In practice, we have these observations:
  - Nonlinearity usually appears locally among some variables.
  - Real invariants (especially for programs) usually contain very few monomials.
- Question: How to exploit such sparsity information in template setting and constraint solving?
- Our recent attempt: combine **treewidth** techniques (sparsity from graph theory) with **quantifier elimination**. $\Longrightarrow$ TACAS'26.

Future Direction: use sparsity to conquer scalability

- Reasoning about nonlinear arithmetic + quantifier alternation is inherently hard.
  - It is unrealistic to expect to find an general and efficient solution.
- In practice, we have these observations:
  - Nonlinearity usually appears locally among some variables.
  - Real invariants (especially for programs) usually contain very few monomials.
- Question: How to exploit such sparsity information in template setting and constraint solving?
- Our recent attempt: combine **treewidth** techniques (sparsity from graph theory) with **quantifier elimination**. $\Longrightarrow$ TACAS'26.

Future Direction: use sparsity to conquer scalability

- Reasoning about nonlinear arithmetic + quantifier alternation is inherently hard.
  - It is unrealistic to expect to find an general and efficient solution.
- In practice, we have these observations:
  - Nonlinearity usually appears locally among some variables.
  - Real invariants (especially for programs) usually contain very few monomials.
- Question: How to exploit such sparsity information in template setting and constraint solving?
- Our recent attempt: combine **treewidth** techniques (sparsity from graph theory) with **quantifier elimination**. $\Longrightarrow$ TACAS'26.

Future Direction: a unified framework

In mathematics, an invariant is a property of a mathematical object which remains unchanged after operations or transformations.

- The constraint-based idea provides a way to encoding different types of operations and properties.
  - **Programs**: invariants, variants (ranking functions).
  - **ODE/Hybrid systems**: barrier certificates, Lyapunov functions
  - **Stochastic systems**: stochastic invariants, ranking supermartingales
  - **Control systems**: control barrier functions, control Lyapunov functions
- Challenge: Can we design a unified and extendable framework for synthesizing such certificates?

Future Direction: a unified framework

In mathematics, an invariant is a property of a mathematical object which remains unchanged after operations or transformations.

- The constraint-based idea provides a way to encoding different types of operations and properties.
  - **Programs**: invariants, variants (ranking functions).
  - **ODE/Hybrid systems**: barrier certificates, Lyapunov functions
  - **Stochastic systems**: stochastic invariants, ranking supermartingales
  - **Control systems**: control barrier functions, control Lyapunov functions
- Challenge: Can we design a unified and extendable framework for synthesizing such certificates?

Future Direction: a unified framework

In mathematics, an invariant is a property of a mathematical object which remains unchanged after operations or transformations.

- The constraint-based idea provides a way to encoding different types of operations and properties.
  - **Programs**: invariants, variants (ranking functions).
  - **ODE/Hybrid systems**: barrier certificates, Lyapunov functions
  - **Stochastic systems**: stochastic invariants, ranking supermartingales
  - **Control systems**: control barrier functions, control Lyapunov functions
- Challenge: Can we design a unified and extendable framework for synthesizing such certificates?

Experience and Published Papers

UCAS from 2016 to 2026:

- **Hao Wu***, Jiyu Zhu*, Amir Goharshady, Jie An, Bican Xia, and Naijun Zhan. Quantifier Elimination Meets Treewidth. **TACAS'2026**.
- Yihao Yin, **Hao Wu**, Wan Liu, Shuling Wang, Xiong Xu, Wang Lin, Fanjiang Xu, and Naijun Zhan. Formal Design of Safety-Critical Systems with MARs. **JSA'2026**.
- **Hao Wu**, Qiuye Wang, Bai Xue, Naijun Zhan, Lihong Zhi, and Zhi-Hong Yang. Synthesizing Invariants for Polynomial Programs by Semidefinite Programming. **TOPLAS'2025**.
- Xiong Xu, Jean-Pierre Talpin, Shuling Wang, **Hao Wu**, Bohua Zhan, Xinxin Liu, and Naijun Zhan. HpC: A Calculus for Hybrid and Mobile Systems. **OOPSLA'2025**.
- Guanhua Lin, **Hao Wu**, and Naijun Zhan. Barrier Certificate Synthesis via Interpolation and Difference-of-Convex Programming. **Festschrift of Prof. Martin Fränzle's 60th birthday, 2025.**
- **Hao Wu**, Shenghua Feng, Ting Gan, Jie Wang, Bican Xia, and Naijun Zhan. On Completeness of SDP-Based Barrier Certificate Synthesis over Unbounded Domains. **FM'2024**.
- **Hao Wu***, Jie Wang*, Bican Xia, Xiakun Li, Naijun Zhan, and Ting Gan. Nonlinear Craig Interpolant Generation Over Unbounded Domains by Separating Semialgebraic Sets. **FM'2024**.
- Yihao Yin, **Hao Wu**, Shuling Wang, Xiong Xu, Fanjiang Xu, and Naijun Zhan. The Design of Intelligent Temperature Control System of Smart House with MARS. **SETTA'2024**.
- **Hao Wu**, Yu-Fang Chen, Zhilin Wu, Bican Xia, and Naijun Zhan. A decision procedure for string constraints with string/integer conversion and flat regular constraints. **Acta Informatica'2024**.
- 王淑灵, 詹博华, 盛欢欢, **吴昊**, 易士程, 王令泰, 金翔宇, 薛白, 李静辉, 向霜晴, 向展, 毛碧飞. 可信系统性质的分类和形式化研究综述. **软件学报'2022**.
- Xiaohui Bei, Xiaoming Sun, **Hao Wu**†, Jialin Zhang, Zhijie Zhang, and Wei Zi. Cake Cutting on Graphs: A Discrete and Bounded Proportional Protocol. **SODA'2020** (undergraduate period).

Thanks! & Questions?

Experience and Published Papers

UCAS from 2016 to 2026:

- **Hao Wu***, Jiyu Zhu*, Amir Goharshady, Jie An, Bican Xia, and Naijun Zhan. Quantifier Elimination Meets Treewidth. **TACAS'2026**.
- Yihao Yin, **Hao Wu**, Wan Liu, Shuling Wang, Xiong Xu, Wang Lin, Fanjiang Xu, and Naijun Zhan. Formal Design of Safety-Critical Systems with MARs. **JSA'2026**.
- **Hao Wu**, Qiuye Wang, Bai Xue, Naijun Zhan, Lihong Zhi, and Zhi-Hong Yang. Synthesizing Invariants for Polynomial Programs by Semidefinite Programming. **TOPLAS'2025**.
- Xiong Xu, Jean-Pierre Talpin, Shuling Wang, **Hao Wu**, Bohua Zhan, Xinxin Liu, and Naijun Zhan. HpC: A Calculus for Hybrid and Mobile Systems. **OOPSLA'2025**.
- Guanhua Lin, **Hao Wu**, and Naijun Zhan. Barrier Certificate Synthesis via Interpolation and Difference-of-Convex Programming. **Festschrift of Prof. Martin Fränzle's 60th birthday, 2025.**
- **Hao Wu**, Shenghua Feng, Ting Gan, Jie Wang, Bican Xia, and Naijun Zhan. On Completeness of SDP-Based Barrier Certificate Synthesis over Unbounded Domains. **FM'2024**.
- **Hao Wu***, Jie Wang*, Bican Xia, Xiakun Li, Naijun Zhan, and Ting Gan. Nonlinear Craig Interpolant Generation Over Unbounded Domains by Separating Semialgebraic Sets. **FM'2024**.
- Yihao Yin, **Hao Wu**, Shuling Wang, Xiong Xu, Fanjiang Xu, and Naijun Zhan. The Design of Intelligent Temperature Control System of Smart House with MARS. **SETTA'2024**.
- **Hao Wu**, Yu-Fang Chen, Zhilin Wu, Bican Xia, and Naijun Zhan. A decision procedure for string constraints with string/integer conversion and flat regular constraints. **Acta Informatica'2024**.
- 王淑灵, 詹博华, 盛欢欢, **吴昊**, 易士程, 王令泰, 金翔宇, 薛白, 李静辉, 向霜晴, 向展, 毛碧飞. 可信系统性质的分类和形式化研究综述. **软件学报'2022**.
- Xiaohui Bei, Xiaoming Sun, **Hao Wu**†, Jialin Zhang, Zhijie Zhang, and Wei Zi. Cake Cutting on Graphs: A Discrete and Bounded Proportional Protocol. **SODA'2020** (undergraduate period).

Thanks! & Questions?